ICPC Notebook — plantillas probadas (Python + C++)

Plantilla base y reglas del juez

Lo primero que se copia en cualquier problema. Las plantillas del resto del repo **asumen** este encabezado (`bits/stdc++.h`, `using namespace std`, `ll`, `INF`, `MD`).

Archivo	Para qué
`template.cpp`	Arranque en C++: I/O rápido, lectura de n y un vector
`template.py`	Arranque en Python: `readline` rápido, imports útiles
`header.h`	Mismo encabezado, para que los tests lo incluyan (en el concurso NO se usa)

¿Python o C++?

Situación	Lenguaje
Strings / simulación con N ≤ 10^4, enteros gigantes, parsing	Python
N ≥ 10^5, grafos, árboles, DP grandes, estructuras de datos	C++
Dudas	C++: los límites de tiempo se calibran con soluciones compiladas

Operaciones por segundo aproximadas: C++ ≈ 10^8, Python ≈ 10^7 (menos con bucles anidados).

N máximo	Complejidad que pasa
≤ 12	O(N!)
≤ 20–25	O(2^N · N)
≤ 500	O(N^3)
≤ 5 000	O(N^2)
≤ 10^6	O(N log N)
≤ 10^8	O(N)
> 10^9	O(√N) o O(log N)

Compilar y probar en local

```
g++ -std=c++20 -O2 -Wall A.cpp -o A
./A < ejemplo.in
python3 A.py < ejemplo.in
```

Antes de CADA envío (juez BOCA)

Ver `09-estrategia` para la lista completa. Lo mínimo:

1. Archivo con el nombre que pide el enunciado (`A.cpp`, `A.py3`); en Java/Kotlin sin `package`.
2. Nada en `cerr`/stderr: cuenta en el límite de salida y puede dar Runtime Error.
3. Un espacio entre valores, `\n` al final de cada línea.
4. `ll` en sumas, productos y distancias.

Sintaxis: "quiero hacer X" → código

Para cuando sabes **qué** hacer pero no recuerdas **cómo se escribe**. Busca la intención en la columna izquierda.

Archivo	Contenido
Este README	Tablas rápidas Python ↔ C++ (leer, imprimir, strings, listas, diccionarios, números)
`python.md`	Python por tema, con trampas del juez
`cpp.md`	C++ y la STL por tema, con trampas
`ejemplo_record.py`	Ejercicio resuelto: contar días con un nuevo máximo (leer n + lista)

Leer líneas con espacios que importan (por ejemplo, la segunda línea es `[ abc]`):

```
import sys
texto = sys.stdin.readline().rstrip("\n")   # NO uses .strip(): borraria espacios utiles
f, guia = input().split(" ", 1)             # separa solo en el PRIMER espacio
```

Busca la intención en la columna izquierda. En C++ se asume `#include <bits/stdc++.h>`, `using namespace std;` y `typedef long long ll;` (ver [C1]).

Q1 — Leer entrada

Quiero…	Python	C++
Un entero	`n = int(input())`	`int n; cin >> n;`
Dos enteros en una línea	`x, y = map(int, input().split())`	`cin >> x >> y;`
Una línea de N enteros → lista	`a = list(map(int, input().split()))`	`vector<ll> a(n); for (auto &x : a) cin >> x;`
Una palabra / string sin espacios	`s = input().strip()`	`string s; cin >> s;`
Una línea completa con espacios	`line = input()`	`getline(cin, line);` (si antes usaste `cin >>`, llama `getline` una vez extra para comer el salto)
Matriz R×C de números	`m = [list(map(int, input().split())) for _ in range(R)]`	`vector<vector<int>> m(R, vector<int>(C)); for (auto &f : m) for (auto &x : f) cin >> x;`
Grilla de caracteres	`g = [input().strip() for _ in range(R)]`	`vector<string> g(R); for (auto &f : g) cin >> f;`
Todo lo que queda (hasta EOF)	`rest = sys.stdin.read().split()`	`while (cin >> x) { ... }`
Leer rápido (N ≥ 10^5)	`import sys; input = sys.stdin.readline` (luego usa `.strip()` en strings)	`ios::sync_with_stdio(false); cin.tie(nullptr);`

Q2 — Imprimir

Quiero…	Python	C++
Lista separada por espacios	`print(*a)`	`for (int i = 0; i < n; i++) cout << a[i] << " \n"[i == n - 1];`
Un valor por línea	`print(*a, sep="\n")`	`for (auto x : a) cout << x << "\n";`
Decimal con 2 cifras	`print(f"{x:.2f}")`	`cout << fixed << setprecision(2) << x << "\n";`
Muchas líneas rápido	`sys.stdout.write("\n".join(out) + "\n")`	usa `"\n"`, nunca `endl`
Sí/No	`print("YES" if ok else "NO")`	`cout << (ok ? "YES" : "NO") << "\n";`

Q3 — Strings

Quiero…	Python	C++
Contar ocurrencias de un carácter	`s.count("a")`	`count(s.begin(), s.end(), 'a')`
Posición de una subcadena (−1 si no está)	`s.find("na")`	`s.find("na")` (devuelve `string::npos` si no está)
¿Contiene?	`"na" in s`	`s.find("na") != string::npos`
Subcadena desde i, largo k	`s[i:i+k]`	`s.substr(i, k)`
Invertir	`s[::-1]`	`reverse(s.begin(), s.end());`
Reemplazar	`s.replace("a", "o")`	(a mano con un bucle)
Dividir por espacios / por coma	`s.split()` / `s.split(",")`	`stringstream ss(line); while (ss >> w) ...`
Unir lista de strings	`" ".join(lst)`	bucle con +=
String → número	`int(s)`	`stoi(s)` / `stoll(s)`
Número → string	`str(x)`	`to_string(x)`
Dígito carácter → valor	`int(c)` u `ord(c) - 48`	`c - '0'`
Letra → índice 0..25	`ord(c) - ord('a')`	`c - 'a'`
Mayúsculas / minúsculas	`s.upper()` / `s.lower()`	`toupper(c)` / `tolower(c)`

Quiero…	Python	C++
¿Es dígito / letra?	`c.isdigit()` / `c.isalpha()`	`isdigit(c)` / `isalpha(c)`
Frecuencia de cada letra	`Counter(s)`	`int f[26] = {}; for (char c : s) f[c - 'a']++;`
Caracteres ordenados	`"".join(sorted(s))`	`sort(s.begin(), s.end());`

Q4 — Listas / vectores

Quiero…	Python	C++
Contar cuántas veces aparece x	`a.count(x)`	`count(a.begin(), a.end(), x)`
Máximo / mínimo / suma	`max(a)`, `min(a)`, `sum(a)`	`*max_element(a.begin(), a.end())`, `accumulate(a.begin(), a.end(), 0LL)`
Índice del máximo	`a.index(max(a))`	`max_element(a.begin(), a.end()) - a.begin()`
Ordenar ascendente / descendente	`a.sort()` / `a.sort(reverse=True)`	`sort(a.begin(), a.end());` / `sort(a.rbegin(), a.rend());`
Ordenar por 2 criterios (1.º desc, 2.º asc)	`a.sort(key=lambda p: (-p[0], p[1]))`	`sort(..., [](auto &x, auto &y){ if (x.first != y.first) return x.first > y.first; return x.second < y.second; });`
Quitar duplicados	`sorted(set(a))`	`sort(...); a.erase(unique(a.begin(), a.end()), a.end());`
Último elemento	`a[-1]`	`a.back()`
Recorrer con índice	`for i, v in enumerate(a):`	`for (int i = 0; i < n; i++)`
Recorrer dos listas a la vez	`for u, v in zip(a, b):`	`for (int i = 0; i < n; i++) { a[i]; b[i]; }`
Primer índice con a[i] ≥ x (lista ordenada)	`bisect_left(a, x)`	`lower_bound(a.begin(), a.end(), x) - a.begin()`
Primer índice con a[i] > x	`bisect_right(a, x)`	`upper_bound(a.begin(), a.end(), x) - a.begin()`
Lista de N ceros / matriz	`[0]*n` / `[[0]*C for _ in range(R)]`	`vector<ll> v(n, 0);` / `vector<vector<int>> m(R, vector<int>(C, 0));`
¿Alguno / todos cumplen?	`any(x > 7 for x in a)` / `all(...)`	`any_of(a.begin(), a.end(), [](ll x){ return x > 7; })`

Q5 — Contar, agrupar, buscar rápido

Quiero…	Python	C++
Diccionario contador	`d = defaultdict(int); d[k] += 1`	`map<string,int> d; d[k]++;`
Agrupar valores por clave	`d = defaultdict(list); d[k].append(v)`	`map<int, vector<int>> d; d[k].push_back(v);`
Lo más frecuente	`Counter(a).most_common(1)`	recorrer el map buscando el mayor
¿Ya vi este valor?	`vis = set(); x in vis`	`set<int> vis; vis.count(x)`
Sacar siempre el menor	`heapq.heappush(h, x); heapq.heappop(h)`	`priority_queue<ll, vector<ll>, greater<ll>> pq;`
Sacar siempre el mayor	push `-x`, pop y niega	`priority_queue<ll> pq;`
Cola por ambos lados	deque: `append`, `appendleft`, `pop`, `popleft`	deque: `push_back`, `push_front`, `pop_back`, `pop_front`

Q6 — Números

Quiero…	Python	C++
División entera / resto	`a // b`, `a % b`	`a / b`, `a % b` (¡con negativos difiere! ver [P7])
Redondear hacia arriba a/b (positivos)	`(a + b - 1) // b`	`(a + b - 1) / b`
MCD / MCM	`math.gcd(a, b)`, `math.lcm(a, b)`	`gcd(a, b)`, `lcm(a, b)`
Raíz entera	`math.isqrt(n)`	`ll r = sqrtl(n); while (r*r > n) r--; while ((r+1)*(r+1) <= n) r++;`

Quiero…	Python	C++
Potencia mod m	`pow(b, e, m)`	`pw(b, e, m)` en [M1]
Combinaciones n en k (exacto)	`math.comb(n, k)`	`C(n, k)` mod p en [M1]
Binario	`bin(x)[2:]`, `int("1010", 2)`	`bitset<32>(x).to_string()`, `stoll("1010", nullptr, 2)`
Contar bits en 1	`x.bit_count()` (Python ≥ 3.10) o `bin(x).count("1")`	`__builtin_popcountll(x)`
Infinito	`float('inf')` o `10**18`	`const ll INF = 4e18;`

Python por tema

[P1] Plantilla base — Cópiala al empezar cualquier problema en Python.

```
import sys
from collections import Counter, defaultdict, deque
import heapq, bisect, math, itertools
input = sys.stdin.readline          # lectura rapida
sys.setrecursionlimit(10**6)        # solo si usas recursion

def main():
    n = int(input())
    a = list(map(int, input().split()))
    s = input().strip()             # .strip() OBLIGATORIO con readline
    print(*a)

main()
```

Trampa: con `input = sys.stdin.readline` el string trae el `\n` al final; `int()` y `split()` lo ignoran, pero `s` sin `.strip()` no.

[P2] Ejemplo corregido — contar días en que aparece un nuevo máximo (el ejercicio de hoy).

```
n = int(input())
a = list(map(int, input().split()))   # toda la linea -> lista de enteros, una sola vez
mx, dias = a[0], 1
for v in a[1:]:
    if v > mx:
        mx = v
        dias += 1
print(dias)
```

[P3] Strings — son inmutables: para modificar, pasa a lista y luego `"".join(lista)`.

```
s = "banana"
s.count("a")      # 3
s.find("n")       # 2   (-1 si no esta)
s.rfind("n")      # 4   (ultima aparicion)
s.index("n")      # 2   (ValueError si no esta)
s[1:4]            # "ana"  (fin exclusivo)
s[::-1]           # "ananab"
s.replace("a","o")# "bonono"
s.startswith("ba") / s.endswith("na")
s.split() / s.split(",") / " ".join(lst)
s.strip() / s.upper() / s.lower() / s.isdigit() / s.isalpha()
"12".zfill(4)     # "0012"
ord("a") -> 97    chr(98) -> "b"
lista = list(s); lista[0] = "B"; s = "".join(lista)
```

[P4] Listas

```
a.append(x); a.pop(); a.pop(0)   # pop(0) es O(n): usa deque
a.count(x); a.index(x); a.reverse(); a.sort()
sorted(a, reverse=True)
sorted(pares, key=lambda p: (-p[0], p[1]))   # 1.o desc, 2.o asc
max(range(n), key=lambda i: a[i])            # indice del maximo
max(words, key=len)
[x*x for x in a if x % 2 == 0]               # comprension
m = [[0]*C for _ in range(R)]                # NUNCA [[0]*C]*R (filas compartidas)
pre = [0]
for v in a: pre.append(pre[-1] + v)          # suma a[l..r] = pre[r+1] - pre[l]
```

[P5] Diccionarios, sets y contadores

```
d = {}; d[k] = d.get(k, 0) + 1
d = defaultdict(int); d[k] += 1          # no falla si k no existe
g = defaultdict(list); g[u].append(v)    # lista de adyacencia
c = Counter(s); c["a"]; c.most_common(2)  # [("a",3),("n",2)]
for k, v in d.items(): ...
for k in sorted(d): ...                  # claves en orden
st = set(); st.add(x); st.discard(x); x in st   # O(1)
```

[P6] Estructuras

```
q = deque([s]); q.append(x); q.appendleft(x); q.popleft(); q.pop()
h = []; heapq.heappush(h, (dist, nodo)); d, u = heapq.heappop(h)   # min-heap
heapq.heappush(h, -x); mayor = -heapq.heappop(h)                   # max-heap
b = [1,3,3,7]
bisect.bisect_left(b, 3)   # 1  primer indice con b[i] >= 3
bisect.bisect_right(b, 3)  # 3  primer indice con b[i] > 3
bisect.insort(b, 5)        # insertar manteniendo orden (O(n))
```

[P7] Matemática e itertools

```
math.gcd(a,b); math.lcm(a,b); math.isqrt(n); math.comb(n,k); math.perm(n,k);
math.factorial(n)
pow(b, e, m)          # potencia modular
pow(a, -1, m)         # inverso modular (m primo o coprimo)
divmod(17, 5)         # (3, 2)
-7 // 2  -> -4        # Python redondea hacia -inf
-7 % 3   -> 2         # resto siempre >= 0 (en C++ da -1)
bin(10)[2:]; int("1010", 2); int("ff", 16); hex(255)
itertools.permutations(a, 2); itertools.combinations(a, 2)
itertools.product([0,1], repeat=n)   # todas las mascaras
itertools.accumulate(a)              # sumas prefijas
```

[P8] Trampas de Python en el juez

Trampa	Solución
TLE con N ≥ 10^5 y bucles anidados	Pasa a C++
Recursión profunda (DFS) revienta	`sys.setrecursionlimit` no basta: usa pila explícita o C++
`print` dentro de un bucle grande es lento	Junta en lista y haz un solo `sys.stdout.write`
`[[0]*C]*R` comparte filas	`[[0]*C for _ in range(R)]`
Comparar strings numéricos (`"10" < "9"` es True)	Convierte con `int()` antes
Nombre del archivo	Debe ser la letra del problema: `A.py3` (ver [J])

C++ y la STL por tema

[C1] Plantilla base — Todas las plantillas de [A]–[GE] asumen este encabezado. Compila con `g++ -std=c++20 -O2 -Wall A.cpp -o A`.

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll INF = 4e18;
const ll MD = 1e9 + 7;

int main(){
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    vector<ll> a(n);
    for (auto &x : a) cin >> x;
    // ... resolver ...
    cout << "respuesta" << "\n";
}
```

[C2] Strings

```
string s; cin >> s;                    // una palabra
string line; getline(cin, line);       // linea completa (tras cin >>, llama getline 2 veces)
s.size(); s[i]; s += 'x'; s += "abc"; s.push_back('x'); s.pop_back();
s.substr(i, k);                        // desde i, largo k
s.find("na");                          // string::npos si no esta
count(s.begin(), s.end(), 'a');
reverse(s.begin(), s.end()); sort(s.begin(), s.end());
stoi(s); stoll(s); to_string(x);
c - '0'; c - 'a'; (char)('a' + k); isdigit(c); isalpha(c); toupper(c);
stringstream ss(line); string w; while (ss >> w) palabras.push_back(w);
```

[C3] Vectores y ordenamiento

```
vector<int> v(n, 0); v.push_back(x); v.pop_back(); v.back(); v.size(); v.clear();
vector<vector<int>> m(R, vector<int>(C, 0));
sort(v.begin(), v.end());              // ascendente
sort(v.rbegin(), v.rend());            // descendente
sort(p.begin(), p.end(), [](auto &x, auto &y){
    if (x.first != y.first) return x.first > y.first;   // 1.o desc
    return x.second < y.second; });                    // 2.o asc
*max_element(v.begin(), v.end()); max_element(v.begin(), v.end()) - v.begin();
accumulate(v.begin(), v.end(), 0LL);   // 0LL, no 0, o se desborda
lower_bound(v.begin(), v.end(), x) - v.begin();   // primer >= x
upper_bound(v.begin(), v.end(), x) - v.begin();   // primer > x
v.erase(unique(v.begin(), v.end()), v.end());     // tras sort: quitar duplicados
next_permutation(v.begin(), v.end());             // do { ... } while
(next_permutation(...));
pair<int,int> pr = {1, 2}; pr.first; pr.second;
for (auto &[k, val] : pares) { ... }              // desempaquetar
```

[C4] set, multiset, map

```
set<int> st; st.insert(x); st.erase(x); st.count(x);   // ordenado, sin repetidos, O(log n)
*st.begin(); *st.rbegin();                             // menor / mayor
*st.lower_bound(x);                                    // menor elemento >= x (verifica != st.end())
multiset<int> ms; ms.erase(ms.find(x));                // borra UNA copia (ms.erase(x) borra todas)
map<string,int> mp; mp[k]++; mp.count(k);              // ordenado por clave
for (auto &[k, v] : mp) { ... }
unordered_map<int,int> um;                             // O(1) promedio, sin orden
```

[C5] Colas, pilas, heaps

```
queue<int> q; q.push(x); q.front(); q.pop(); q.empty();
stack<int> sk; sk.push(x); sk.top(); sk.pop();
deque<int> dq; dq.push_front(x); dq.push_back(x); dq.front(); dq.back();
priority_queue<int> mx;                                  // saca el MAYOR
priority_queue<int, vector<int>, greater<int>> mn;       // saca el MENOR
priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;   // Dijkstra
```

[C6] Bits y utilidades

```
__builtin_popcountll(x);      // bits en 1
__lg(x);                      // posicion del bit mas alto (floor log2)
x & (1 << i);                 // bit i encendido?
for (int mask = 0; mask < (1 << n); mask++)   // todos los subconjuntos
gcd(a, b); lcm(a, b);
bitset<32>(x).to_string();
cout << fixed << setprecision(6) << x;
min({a, b, c}); max({a, b, c}); swap(a, b);
```

[C7] Trampas de C++

Trampa	Solución
Desborde: `int` llega a ~2·10^9	Usa `ll` para sumas, productos y distancias; `1LL * a * b`
`endl` en bucles grandes	Usa `"\n"`
`cerr` de depuración en el envío	Bórralo: cuenta en el límite de salida (ver [J])
`-7 % 3 == -1`	`((x % m) + m) % m`
`v.size() - 1` con vector vacío	Es unsigned: da un número gigante. Usa `(int)v.size() - 1`

Trampa	Solución
`accumulate(..., 0)`	`0LL`
Arreglo local enorme (`int a[1000000]` en `main`)	Decláralo global o usa `vector`
Comparar `double` con `==`	`fabs(a - b) < 1e-9`, o trabaja en enteros

Básicos: prefijos, búsqueda binaria, dos punteros, intervalos

Técnicas que aparecen dentro de casi cualquier problema medio.

Archivo	Qué hace	Complejidad
`prefijos_2d.cpp`	Suma de cualquier submatriz	O(1) por consulta
`busqueda_binaria_respuesta.cpp`	Menor x con `ok(x)` verdadero	O(log rango · costo de ok)
`dos_punteros.cpp`	Ventana más larga con suma ≤ K	O(n)
`intervalos_union.cpp`	Longitud total cubierta por intervalos reales	O(n log n)
`intervalos_fusionar.cpp`	Fusionar intervalos enteros solapados	O(n log n)

[A1] Sumas prefijas 1D — Cuándo: muchas consultas "suma de a[l..r]". Trampa: el arreglo `p` mide n+1.

```
vector<ll> p(n + 1, 0);
for (int i = 0; i < n; i++) p[i + 1] = p[i] + a[i];
// suma a[l..r] (0-indexado, inclusivo) = p[r + 1] - p[l]
```

[A2] Sumas prefijas 2D — Cuándo: suma de submatrices. Trampa: índices 1-indexados en `rect`.

```
// Sumas prefijas 2D (indices 1..n, 1..m)
vector<vector<ll>> pref2D(vector<vector<ll>>& a){ int n = a.size(), m = a[0].size();
    vector<vector<ll>> p(n + 1, vector<ll>(m + 1, 0));
    for (int i = 1; i <= n; i++) for (int j = 1; j <= m; j++) p[i][j] = a[i-1][j-1] + p[i-1]
[j] + p[i][j-1] - p[i-1][j-1];
    return p; }
ll rect(vector<vector<ll>>& p, int r1, int c1, int r2, int c2){ // inclusivo, 1-indexado
    return p[r2][c2] - p[r1-1][c2] - p[r2][c1-1] + p[r1-1][c1-1]; }
```

[A3] Búsqueda binaria sobre la respuesta — Cuándo: "el mínimo X tal que se puede…" y si X sirve, X+1 también sirve. Trampa: `ok` debe ser monótona (F…F T…T); para "el máximo X" invierte la condición.

```
// Busqueda binaria sobre la respuesta: menor x en [lo,hi] con ok(x)=true (ok monotona
F..FT..T)
template<class F> ll firstTrue(ll lo, ll hi, F ok){ while (lo < hi){ ll mid = lo + (hi - lo)
/ 2; if (ok(mid)) hi = mid; else lo = mid + 1; } return lo; }
```

[A4] Dos punteros / ventana deslizante — Cuándo: subarreglo más largo/corto que cumple algo y la condición se mantiene al encoger. Trampa: requiere valores ≥ 0 (si hay negativos usa prefijos + map).

```
// Dos punteros: subarreglo mas largo con suma <= K (valores >= 0)
int longestAtMost(vector<ll>& a, ll K){ int best = 0; ll s = 0;
    for (int l = 0, r = 0; r < (int)a.size(); r++){ s += a[r]; while (s > K) s -= a[l++];
best = max(best, r - l + 1); }
    return best; }
```

[IV1] Longitud de la unión — Cuándo: "cuánto del eje queda cubierto" (sombras, rangos de luz, tramos). O(n log n). Probado con los 3 ejemplos de LAR 2024 J (Jigsaw of Shadows), donde cada intervalo es [X, X + H / tan(θ)].

```
// Longitud de la union de intervalos [l, r]
// Requiere: reales/base_flotante.cpp (ld)
ld unionLength(vector<pair<ld,ld>> v){
    sort(v.begin(), v.end()); ld tot = 0, curL = 0, curR = -1e30; bool open = false;
    for (auto [l, r] : v){
        if (!open || l > curR){ if (open) tot += curR - curL; curL = l; curR = r; open =
true; }
        else curR = max(curR, r); }
    if (open) tot += curR - curL; return tot; }
```

[IV2] Fusionar intervalos enteros — Devuelve la lista de bloques disjuntos. Por defecto une [1,3] y [4,6] (se tocan); quita el `+ 1` si no deben unirse.

```
// Fusionar intervalos enteros [l, r] que se solapan o tocan
vector<pair<ll,ll>> mergeIntervals(vector<pair<ll,ll>> v){
    sort(v.begin(), v.end()); vector<pair<ll,ll>> res;
    for (auto [l, r] : v){ if (!res.empty() && l <= res.back().second + 1) res.back().second
= max(res.back().second, r); else res.push_back({l, r}); }
    return res; } // quita el +1 si [1,3] y [4,6] NO deben unirse
```

Otros patrones de intervalos: máximo de intervalos solapados en un punto → eventos (+1 en l, −1 en r+1), ordenar y barrer; máximo de intervalos disjuntos → ordenar por extremo derecho y tomar greedy.

Tests: `tests/test_basicos.cpp`.

Grafos y árboles

El bloque con más peso en el regional latinoamericano: entre 3 y 5 problemas por edición. Todo en C++ y 0-indexado (si la entrada es 1-indexada, resta 1 al leer).

Archivo	Qué hace	Complejidad
`bfs.cpp`	Distancias con aristas de costo 1	O(n + m)
`bfs_grilla.cpp`	BFS en un laberinto (# = pared)	O(R·C)
`componentes.cpp`	Componentes conexas con DFS iterativo	O(n + m)
`dijkstra.cpp`	Caminos más cortos con pesos ≥ 0 (opción de ignorar un nodo)	O(m log n)
`en_camino_mas_corto.cpp`	¿v está en algún camino más corto de s a t?	O(n)
`toposort.cpp`	Orden topológico (Kahn); detecta ciclos	O(n + m)
`dsu.cpp`	Union-Find	~O(1) por operación
`kruskal.cpp`	Árbol de expansión mínima	O(m log m)
`arbol_dfs_puentes.cpp`	Árbol DFS, back-edges, puentes, articulaciones	O(n + m)
`lca.cpp`	LCA, distancia y k-ésimo ancestro en árboles	O(log n) por consulta
`rerooting.cpp`	DP "para cada nodo como raíz"	O(n)
`hopcroft_karp.cpp`	Matching bipartito máximo	O(m √n)
`dinic.cpp`	Flujo máximo / corte mínimo	O(n² m) peor caso, rápido en la práctica

[G1] Leer un grafo — Cuándo: siempre. Trampa: no dirigido = agrega la arista en ambos sentidos.

```
int n, m; cin >> n >> m;
vector<vector<int>> adj(n);                    // sin pesos
vector<vector<pair<int,ll>>> wadj(n);          // con pesos (vecino, peso)
for (int i = 0; i < m; i++){
    int u, v; ll w; cin >> u >> v >> w; u--; v--;
    adj[u].push_back(v); adj[v].push_back(u);
    wadj[u].push_back({v, w}); wadj[v].push_back({u, w});
}
```

[G2] BFS (distancias sin peso) — Cuándo: camino más corto con aristas de costo 1. Trampa: marca `d[v]` al encolar, no al sacar.

```
// BFS
vector<int> bfs(int s, vector<vector<int>>& adj){
    vector<int> d(adj.size(), -1); queue<int> q; d[s] = 0; q.push(s);
    while (!q.empty()){ int u = q.front(); q.pop();
        for (int v : adj[u]) if (d[v] == -1){ d[v] = d[u] + 1; q.push(v); } }
    return d;
}
```

[G3] BFS en grilla — Cuándo: laberintos, # = pared. Trampa: revisa límites ANTES de leer `g[nr][nc]`.

```cpp
// BFS grid
int dr[4] = {-1,1,0,0}, dc[4] = {0,0,-1,1};
vector<vector<int>> bfsGrid(vector<string>& g, int sr, int sc){
    int R = g.size(), C = g[0].size();
    vector<vector<int>> d(R, vector<int>(C, -1)); queue<pair<int,int>> q;
    d[sr][sc] = 0; q.push({sr, sc});
    while (!q.empty()){ auto [r, c] = q.front(); q.pop();
        for (int k = 0; k < 4; k++){ int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
            if (g[nr][nc] == '#' || d[nr][nc] != -1) continue;
            d[nr][nc] = d[r][c] + 1; q.push({nr, nc}); } }
    return d;
}
```

[G4] DFS iterativo / componentes conexas — Cuándo: contar componentes, alcanzabilidad. Trampa: DFS recursivo con N = 10^5 en línea puede desbordar la pila; esta versión no.

```cpp
// DFS iterativo (componentes)
vector<int> components(vector<vector<int>>& adj, int &cnt){
    int n = adj.size(); vector<int> comp(n, -1); cnt = 0;
    for (int s = 0; s < n; s++) if (comp[s] == -1){
        stack<int> st; st.push(s); comp[s] = cnt;
        while (!st.empty()){ int u = st.top(); st.pop();
            for (int v : adj[u]) if (comp[v] == -1){ comp[v] = cnt; st.push(v); } }
        cnt++; }
    return comp;
}
```

[G5] Dijkstra (+ omitir un nodo) — Cuándo: caminos más cortos con pesos ≥ 0. `banned` sirve para el truco "¿todos los caminos óptimos pasan por G?": corre con y sin G y compara. Trampa: `if (du > d[u]) continue;` es obligatorio.

```cpp
// Dijkstra (banned = nodo a ignorar, -1 = ninguno)
vector<ll> dijkstra(int s, vector<vector<pair<int,ll>>>& adj, int banned = -1){
    vector<ll> d(adj.size(), INF);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    d[s] = 0; pq.push({0, s});
    while (!pq.empty()){ auto [du, u] = pq.top(); pq.pop();
        if (du > d[u]) continue;
        for (auto [v, w] : adj[u]){ if (v == banned) continue;
            if (d[u] + w < d[v]){ d[v] = d[u] + w; pq.push({d[v], v}); } } }
    return d;
}
```

[G6] Grafo invertido + varias búsquedas — Cuándo: "¿qué nodos llegan a X?" o "¿existe un v alcanzable por p, a y b a la vez?". En vez de un BFS por nodo (O(n²) BFS), invierte aristas y haz un BFS desde cada nodo especial.

```cpp
vector<vector<int>> radj(n);
for (int u = 0; u < n; u++) for (int v : adj[u]) radj[v].push_back(u);
auto dp = bfs(p, radj), da = bfs(a, radj), db = bfs(b, radj);
for (int v = 0; v < n; v++) if (dp[v] != -1 && da[v] != -1 && db[v] != -1) { /* v llega a p, a y b */ }
```

[G7] Orden topológico (Kahn) + DP en DAG — Cuándo: dependencias, "camino más largo", contar caminos en un DAG. Trampa: si `order.size() < n` hay ciclo.

```cpp
vector<int> toposort(vector<vector<int>>& adj){
    int n = adj.size(); vector<int> indeg(n, 0), order;
    for (int u = 0; u < n; u++) for (int v : adj[u]) indeg[v]++;
    queue<int> q; for (int u = 0; u < n; u++) if (!indeg[u]) q.push(u);
    while (!q.empty()){ int u = q.front(); q.pop(); order.push_back(u);
        for (int v : adj[u]) if (--indeg[v] == 0) q.push(v); }
    return order;
}
// DP: camino mas largo
vector<int> lp(n, 0);
for (int u : order) for (int v : adj[u]) lp[v] = max(lp[v], lp[u] + 1);
```

[G8] DSU (Union-Find) — Cuándo: unir grupos y preguntar "¿están juntos?", Kruskal. Trampa: compara siempre `find(a) == find(b)`, nunca `p[a] == p[b]`.

```cpp
// DSU
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x){ return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b){ a = find(a); b = find(b); if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b); p[b] = a; sz[a] += sz[b]; return true; }
};
```

[G9] Árbol DFS: back-edges, puentes, articulaciones — Cuándo: ciclos que no desconectan, aristas críticas, biconexidad. `back` guarda (descendiente, ancestro); `dep` es la profundidad. Trampa: recursivo; N ≤ ~2·10^5 suele estar bien.

```cpp
// DFS tree: profundidad, back-edges, puentes y articulaciones
struct DfsTree {
    int n, timer = 0; vector<vector<pair<int,int>>> adj; // (vecino, id arista)
    vector<int> tin, low, dep, par; vector<bool> isArt; vector<int> bridges;
vector<array<int,2>> back;
    DfsTree(int n) : n(n), adj(n), tin(n, -1), low(n), dep(n), par(n, -1), isArt(n) {}
    void addEdge(int u, int v, int id){ adj[u].push_back({v, id}); adj[v].push_back({u, id});
}
    void dfs(int u, int pe){ tin[u] = low[u] = timer++; int ch = 0;
        for (auto [v, id] : adj[u]){ if (id == pe) continue;
            if (tin[v] != -1){ low[u] = min(low[u], tin[v]); if (tin[v] < tin[u])
back.push_back({u, v}); }
            else { par[v] = u; dep[v] = dep[u] + 1; dfs(v, id); low[u] = min(low[u], low[v]);
ch++;
                if (low[v] > tin[u]) bridges.push_back(id);
                if (pe != -1 && low[v] >= tin[u]) isArt[u] = true; } }
        if (pe == -1 && ch > 1) isArt[u] = true; }
    void run(){ for (int i = 0; i < n; i++) if (tin[i] == -1) dfs(i, -1); }
};
```

[G10] LCA con binary lifting — Cuándo: distancia entre nodos de un árbol, k-ésimo ancestro, saltos sobre caminos. O(N log N) preproceso, O(log N) por consulta.

```cpp
// LCA binary lifting
struct LCA {
    int n, LOG; vector<vector<int>> up; vector<int> dep;
    LCA(vector<vector<int>>& adj, int root = 0) : n(adj.size()) {
        LOG = 1; while ((1 << LOG) < n) LOG++;
        up.assign(LOG, vector<int>(n, root)); dep.assign(n, 0);
        vector<int> st = {root}; vector<bool> vis(n); vis[root] = true;
        while (!st.empty()){ int u = st.back(); st.pop_back();
            for (int v : adj[u]) if (!vis[v]){ vis[v] = true; up[0][v] = u; dep[v] = dep[u] +
1; st.push_back(v); } }
        for (int k = 1; k < LOG; k++) for (int v = 0; v < n; v++) up[k][v] = up[k-1][up[k-1]
[v]];
    }
    int kth(int v, int k){ for (int i = 0; i < LOG; i++) if (k >> i & 1) v = up[i][v]; return
v; }
    int lca(int a, int b){ if (dep[a] < dep[b]) swap(a, b); a = kth(a, dep[a] - dep[b]);
        if (a == b) return a;
        for (int k = LOG - 1; k >= 0; k--) if (up[k][a] != up[k][b]){ a = up[k][a]; b = up[k]
[b]; }
        return up[0][a]; }
    int dist(int a, int b){ return dep[a] + dep[b] - 2 * dep[lca(a, b)]; }
};
```

[G11] Rerooting (DP con cambio de raíz) — Cuándo: "para CADA nodo como raíz, calcula…" en O(N). Patrón: (1) DP de hijos hacia el padre en orden inverso; (2) pasa la respuesta del padre al hijo. Ejemplo: suma de distancias de cada nodo a todos.

```cpp
// Rerooting: suma de distancias desde cada nodo a todos
vector<ll> sumDistAll(vector<vector<int>>& adj){
    int n = adj.size(); vector<int> order, par(n, -1); vector<ll> sz(n, 1), down(n, 0),
ans(n);
    vector<int> st = {0}; par[0] = 0;
    while (!st.empty()){ int u = st.back(); st.pop_back(); order.push_back(u);
        for (int v : adj[u]) if (par[v] == -1){ par[v] = u; st.push_back(v); } }
    par[0] = -1;
    for (int i = n - 1; i >= 0; i--){ int u = order[i];       // 1) hijos -> padre
        for (int v : adj[u]) if (v != par[u]){ sz[u] += sz[v]; down[u] += down[v] + sz[v]; }
}
    ans[0] = down[0];
    for (int u : order) for (int v : adj[u]) if (v != par[u])  // 2) padre -> hijos
        ans[v] = ans[u] - sz[v] + (n - sz[v]);
    return ans;
}
```

Variante "mejor y segundo mejor": en el paso 1 guarda, por nodo, el mejor valor y el segundo mejor que venga de un hijo DISTINTO; en el paso 2, si el mejor del padre viene de v, usa el segundo.

[G12] Matching bipartito (Hopcroft-Karp) — Cuándo: emparejar máximo izquierda↔derecha (tareas/personas, filas/columnas, color de texto/color de fondo). O(E√V). Conjunto independiente máximo en bipartito = V − matching.

```cpp
// Hopcroft-Karp (izq 0..L-1, der 0..R-1)
struct HopcroftKarp {
    int L, R; vector<vector<int>> adj; vector<int> mL, mR, dist;
    HopcroftKarp(int L, int R) : L(L), R(R), adj(L), mL(L, -1), mR(R, -1), dist(L) {}
    void addEdge(int u, int v){ adj[u].push_back(v); }
    bool bfs(){ queue<int> q; bool found = false;
        for (int u = 0; u < L; u++){ if (mL[u] == -1){ dist[u] = 0; q.push(u); } else dist[u]
= -1; }
        while (!q.empty()){ int u = q.front(); q.pop();
            for (int v : adj[u]){ int w = mR[v];
                if (w == -1) found = true; else if (dist[w] == -1){ dist[w] = dist[u] + 1;
q.push(w); } } }
        return found; }
    bool dfs(int u){ for (int v : adj[u]){ int w = mR[v];
            if (w == -1 || (dist[w] == dist[u] + 1 && dfs(w))){ mL[u] = v; mR[v] = u; return
true; } }
        dist[u] = -1; return false; }
    int maxMatching(){ int res = 0; while (bfs()) for (int u = 0; u < L; u++) if (mL[u] == -1
&& dfs(u)) res++; return res; }
};
```

[G13] Flujo máximo / corte mínimo (Dinic) — Cuándo: capacidades, "mínimo de aristas a cortar para separar s de t", asignaciones con límites. Truco: un nodo con capacidad c se divide en `v_in` → `v_out` con capacidad c.

```cpp
// Dinic (flujo maximo / corte minimo)
struct Dinic {
    struct E { int to; ll cap; };
    vector<E> e; vector<vector<int>> g; vector<int> lvl, it;
    Dinic(int n) : g(n), lvl(n), it(n) {}
    void addEdge(int u, int v, ll c){ g[u].push_back(e.size()); e.push_back({v, c});
g[v].push_back(e.size()); e.push_back({u, 0}); }
    bool bfs(int s, int t){ fill(lvl.begin(), lvl.end(), -1); lvl[s] = 0; queue<int> q;
q.push(s);
        while (!q.empty()){ int u = q.front(); q.pop();
            for (int id : g[u]) if (e[id].cap > 0 && lvl[e[id].to] == -1){ lvl[e[id].to] =
lvl[u] + 1; q.push(e[id].to); } }
        return lvl[t] != -1; }
    ll dfs(int u, int t, ll f){ if (u == t) return f;
        for (int &i = it[u]; i < (int)g[u].size(); i++){ int id = g[u][i], v = e[id].to;
            if (e[id].cap > 0 && lvl[v] == lvl[u] + 1){ ll got = dfs(v, t, min(f,
e[id].cap));
                if (got){ e[id].cap -= got; e[id ^ 1].cap += got; return got; } } }
        return 0; }
    ll maxflow(int s, int t){ ll fl = 0; while (bfs(s, t)){ fill(it.begin(), it.end(), 0);
while (ll f = dfs(s, t, INF)) fl += f; } return fl; }
    // corte minimo: tras maxflow, nodos con lvl[v] != -1 estan del lado de s
};
```

Kruskal — Ordena las aristas por peso y agrega cada una si une dos componentes distintas (DSU). Si al final se usaron menos de n − 1 aristas, el grafo no es conexo. Para "repetir el MST y quitar sus aristas" (UCAB 2025 M) se ordena una sola vez y se marca qué aristas ya se usaron. Requiere `dsu.cpp`.

```cpp
// Kruskal: arbol de expansion minima (MST)
// Requiere: dsu.cpp
// aristas: {peso, u, v}. Devuelve {peso total, ids de aristas usadas}; si used.size() < n-1
el grafo no es conexo.
pair<ll, vector<int>> kruskal(int n, vector<array<ll,3>> edges){
    vector<int> id(edges.size()); iota(id.begin(), id.end(), 0);
    sort(id.begin(), id.end(), [&](int a, int b){ return edges[a][0] < edges[b][0]; });
    DSU dsu(n); ll total = 0; vector<int> used;
    for (int i : id) if (dsu.unite(edges[i][1], edges[i][2])){ total += edges[i][0];
used.push_back(i); }
    return {total, used};
}
```

¿v está en un camino más corto de s a t? — Sí, exactamente cuando d_s[v] + d_t[v] = d_s[t]. Con un BFS desde s y otro desde t se marcan todos esos nodos en O(n). Para "caminar juntos el mayor tramo" (UCAB 2025 L) se exige la condición para cada destino y se toma el v con d_s[v] máximo.

```cpp
// "v esta en ALGUN camino mas corto de s a t"  <=>  ds[v] + dt[v] == ds[t]
// ds = distancias desde s, dt = distancias desde t (en grafo NO dirigido; si es dirigido, dt
en el grafo invertido).
// Sirve con BFS (bfs.cpp, -1 = inalcanzable) o Dijkstra (con ll).
vector<bool> onShortestPath(const vector<int>& ds, const vector<int>& dt, int t){
    int n = ds.size(); vector<bool> on(n, false);
    for (int v = 0; v < n; v++) if (ds[v] != -1 && dt[v] != -1 && ds[v] + dt[v] == ds[t])
on[v] = true;
    return on; }
```

Tests: `tests/test_grafos.cpp` (LCA y rerooting contra BFS, matching contra flujo, Kruskal contra Prim).

Estructuras de datos

Para muchas consultas o actualizaciones sobre rangos. Elige con la tabla de abajo.

Necesito…	Archivo
Suma en rango + actualizar un punto; k-ésimo	`fenwick.cpp`
set con "k-ésimo" y "cuántos son < x"	`ordered_set.cpp`
Sumar a un rango + suma de un rango	`segtree_lazy.cpp`
Racha más larga en un rango + invertir un rango	`segtree_rachas_flip.cpp`

Necesito…	Archivo
Mínimo/máximo en rango sin actualizaciones	`sparse_table.cpp`
Máximo de rectas a·x + b	`li_chao.cpp`
Borrar de un círculo en O(1) (Josephus)	`lista_circular.cpp`
Actualizaciones que obligan a recalcular todo	Sqrt batching (esqueleto abajo)

[D1] Fenwick / BIT — Índices 1..n. `kth(k)` = menor posición cuya suma prefija ≥ k (valores ≥ 0). Trampa: ¡nunca uses el índice 0!

```
// Fenwick (indices 1..n) + k-esimo
struct BIT {
    int n; vector<ll> t;
    BIT(int n) : n(n), t(n + 1, 0) {}
    void add(int i, ll v){ for (; i <= n; i += i & -i) t[i] += v; }
    ll sum(int i){ ll s = 0; for (; i > 0; i -= i & -i) s += t[i]; return s; }
    ll sum(int l, int r){ return sum(r) - sum(l - 1); }
    int kth(ll k){ int pos = 0; // menor pos con sum(pos) >= k (valores >= 0)
        for (int pw = 1 << __lg(n); pw; pw >>= 1)
            if (pos + pw <= n && t[pos + pw] < k){ pos += pw; k -= t[pos]; }
        return pos + 1; }
};
```

[D2] Ordered set (PBDS) — Cuándo: un `set` que además responde "k-ésimo" y "posición de x" en O(log n). Trampa: no admite repetidos; para multiset guarda `pair<valor, id_unico>`. Pon estas líneas debajo de `using namespace std;`.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef tree<int, null_type, less<int>, rb_tree_tag, tree_order_statistics_node_update>
ordered_set;
ordered_set os; os.insert(5); os.erase(5);
*os.find_by_order(k);   // k-esimo (0-indexado)
os.order_of_key(x);     // cuantos elementos son < x
```

[D3] Segment tree con lazy (sumar en rango, suma de rango) — Índices 0..n-1, rangos inclusivos. Para mínimo: cambia `+` por `min`, el neutro 0 por INF, y `t[x] += v*(largo)` por `t[x] += v`.

```
// Segment tree con lazy (suma en rango, sumar en rango), indices 0..n-1
struct SegLazy {
    int n; vector<ll> t, lz;
    SegLazy(int n) : n(n), t(4 * n, 0), lz(4 * n, 0) {}
    void push(int x, int l, int r){ if (!lz[x]) return; int m = (l + r) / 2;
        t[2*x] += lz[x] * (m - l + 1); lz[2*x] += lz[x];
        t[2*x+1] += lz[x] * (r - m); lz[2*x+1] += lz[x]; lz[x] = 0; }
    void update(int a, int b, ll v, int x, int l, int r){
        if (b < l || r < a) return;
        if (a <= l && r <= b){ t[x] += v * (r - l + 1); lz[x] += v; return; }
        push(x, l, r); int m = (l + r) / 2;
        update(a, b, v, 2*x, l, m); update(a, b, v, 2*x+1, m + 1, r); t[x] = t[2*x] +
t[2*x+1]; }
    ll query(int a, int b, int x, int l, int r){
        if (b < l || r < a) return 0;
        if (a <= l && r <= b) return t[x];
        push(x, l, r); int m = (l + r) / 2;
        return query(a, b, 2*x, l, m) + query(a, b, 2*x+1, m + 1, r); }
    void update(int a, int b, ll v){ update(a, b, v, 1, 0, n - 1); }
    ll query(int a, int b){ return query(a, b, 1, 0, n - 1); }
};
```

[D4] Sparse table (RMQ en O(1)) — Cuándo: arreglo fijo, miles de consultas min/max. Sirve para min, max, gcd (operaciones idempotentes), no para suma.

```
// Sparse table (minimo en rango O(1))
struct Sparse {
    vector<vector<ll>> st;
    Sparse(vector<ll>& a){ int n = a.size(), K = __lg(n) + 1; st.assign(K, a);
        for (int k = 1; k < K; k++) for (int i = 0; i + (1 << k) <= n; i++)
            st[k][i] = min(st[k-1][i], st[k-1][i + (1 << (k-1))]); }
    ll query(int l, int r){ int k = __lg(r - l + 1); return min(st[k][l], st[k][r - (1 << k)
+ 1]); }
};
```

[D5] Li-Chao tree (máximo de rectas) — Cuándo: insertar rectas a·x+b y preguntar el máximo en un x entero de [lo, hi]; también con cualquier familia donde dos funciones se cruzan a lo sumo una vez (cambia `eval`). Para mínimo: inserta (−a, −b) y niega la respuesta. Trampa: cuida que a·x no desborde `ll`.

```
// Li-Chao tree: maximo de rectas a*x+b en x enteros [lo, hi]
struct LiChao {
    struct Line { ll a, b; ll eval(ll x) const { return a * x + b; } };
    struct Node { Line ln; int l = -1, r = -1; };
    vector<Node> t; ll lo, hi;
    LiChao(ll lo, ll hi) : lo(lo), hi(hi) { t.push_back({{0, LLONG_MIN / 2}}); }
    void add(Line nw, int x, ll l, ll r){
        ll m = l + (r - l) / 2;
        bool lef = nw.eval(l) > t[x].ln.eval(l), mid = nw.eval(m) > t[x].ln.eval(m);
        if (mid) swap(t[x].ln, nw);
        if (l == r) return;
        if (lef != mid){ if (t[x].l == -1){ t[x].l = t.size(); t.push_back({nw}); } else
add(nw, t[x].l, l, m); }
        else { if (t[x].r == -1){ t[x].r = t.size(); t.push_back({nw}); } else add(nw,
t[x].r, m + 1, r); } }
    ll query(ll q, int x, ll l, ll r){ if (x == -1) return LLONG_MIN / 2;
        ll m = l + (r - l) / 2, res = t[x].ln.eval(q);
        if (l == r) return res;
        return max(res, q <= m ? query(q, t[x].l, l, m) : query(q, t[x].r, m + 1, r)); }
    void add(ll a, ll b){ add({a, b}, 0, lo, hi); }
    ll query(ll x){ return query(x, 0, lo, hi); }
};
```

[D6] Sqrt batching (esqueleto) — Cuándo: hay un recalculo global caro (O(N) o O(M), p. ej. DP en orden topológico) y las actualizaciones lo invalidan. Agrupa las actualizaciones en bloques de tamaño B ≈ √Q: entre recalculos, responde con el valor base + corrección por las ≤ B actualizaciones pendientes.

```
const int B = 400;                      // ~ sqrt(numero de operaciones)
vector<Upd> pend;                       // actualizaciones aun no aplicadas al "base"
rebuild();                              // calcula base[] con todo aplicado, O(N)
for (auto &op : ops){
    if (op.esUpdate){
        pend.push_back(op.upd);
        if ((int)pend.size() == B){ aplicar(pend); rebuild(); pend.clear(); }
    } else {
        ll r = base[op.i];
        for (auto &u : pend) r += efecto(u, op.i);   // correccion O(B)
        cout << r << "\n";
    }
}
// total: O((Q/B)*N + Q*B), minimo con B ~ sqrt(N)
```

[D7] Segment tree con merge personalizado — Cuándo: la respuesta de un rango no es una suma, sino algo que se arma uniendo dos mitades (racha más larga, mejor subarreglo, etc.). Cada nodo guarda prefijo, sufijo y mejor racha; al unir, la mejor racha es max(mejor izq, mejor der, sufijo izq + prefijo der). Para "invertir" un rango se guardan los datos de 0s y de 1s y la lazy solo los intercambia. Resuelve UCV 2025 B (Midnight Craving).

```
// Segment tree con merge personalizado: racha mas larga de 0s en un rango + invertir un
rango (lazy)
// Patron general: cada nodo guarda prefijo, sufijo y mejor racha; merge combina sufijo(izq)
+ prefijo(der).
// Se guardan los datos de 0s Y de 1s: invertir = intercambiarlos.
struct RunNode {
    int len = 0, pre[2] = {0, 0}, suf[2] = {0, 0}, best[2] = {0, 0};
};
RunNode mergeRun(const RunNode& a, const RunNode& b){
    if (a.len == 0) return b; if (b.len == 0) return a;
    RunNode r; r.len = a.len + b.len;
    for (int c = 0; c < 2; c++){
        r.pre[c] = a.pre[c] == a.len ? a.len + b.pre[c] : a.pre[c];
        r.suf[c] = b.suf[c] == b.len ? b.len + a.suf[c] : b.suf[c];
        r.best[c] = max({a.best[c], b.best[c], a.suf[c] + b.pre[c]}); }
    return r; }
struct SegRuns {
    int n; vector<RunNode> t; vector<bool> lz;
    SegRuns(const string& s) : n(s.size()), t(4 * s.size()), lz(4 * s.size(), false) {
build(s, 1, 0, n - 1); }
    void build(const string& s, int x, int l, int r){
        if (l == r){ int c = s[l] - '0'; t[x].len = 1; t[x].pre[c] = t[x].suf[c] =
t[x].best[c] = 1; return; }
        int m = (l + r) / 2; build(s, 2*x, l, m); build(s, 2*x+1, m + 1, r); t[x] =
mergeRun(t[2*x], t[2*x+1]); }
    void applyFlip(int x){ swap(t[x].pre[0], t[x].pre[1]); swap(t[x].suf[0], t[x].suf[1]);
swap(t[x].best[0], t[x].best[1]); lz[x] = !lz[x]; }
    void push(int x){ if (lz[x]){ applyFlip(2*x); applyFlip(2*x+1); lz[x] = false; } }
    void flip(int a, int b, int x, int l, int r){
        if (b < l || r < a) return;
        if (a <= l && r <= b){ applyFlip(x); return; }
        push(x); int m = (l + r) / 2; flip(a, b, 2*x, l, m); flip(a, b, 2*x+1, m + 1, r);
t[x] = mergeRun(t[2*x], t[2*x+1]); }
    RunNode query(int a, int b, int x, int l, int r){
        if (b < l || r < a) return RunNode();
        if (a <= l && r <= b) return t[x];
        push(x); int m = (l + r) / 2; return mergeRun(query(a, b, 2*x, l, m), query(a, b,
2*x+1, m + 1, r)); }
    void flip(int a, int b){ flip(a, b, 1, 0, n - 1); }              // 0-indexado, inclusivo
    int longestZeros(int a, int b){ return query(a, b, 1, 0, n - 1).best[0]; }
};
```

[D8] Lista circular con arreglos — `nxt[i]` y `prv[i]`; borrar x es `nxt[prv[x]] = nxt[x]; prv[nxt[x]] = prv[x]`. Para simular círculos donde se elimina gente (UCAB 2025 H). Incluye el Josephus clásico y su fórmula O(n).

```
// Lista doblemente enlazada (circular) con arreglos: borrar en O(1), recorrer vecinos.
// Uso tipico: Josephus / eliminar elementos de un circulo / simular una fila con borrados.
struct CircList {
    vector<int> nxt, prv;
    CircList(int n) : nxt(n), prv(n) { for (int i = 0; i < n; i++){ nxt[i] = (i + 1) % n;
prv[i] = (i - 1 + n) % n; } }
    void erase(int x){ nxt[prv[x]] = nxt[x]; prv[nxt[x]] = prv[x]; }   // x deja el circulo;
nxt[x] sigue valido
};
// Josephus clasico: n personas, se elimina cada k-esima. O(n*k). Devuelve el sobreviviente
(0-indexado).
int josephus(int n, int k){ CircList L(n); int cur = 0, alive = n;
    while (alive > 1){ for (int i = 1; i < k; i++) cur = L.nxt[cur]; int nx = L.nxt[cur];
L.erase(cur); cur = nx; alive--; }
    return cur; }
// Formula O(n) del Josephus clasico: J(1)=0, J(i) = (J(i-1) + k) % i
int josephusFormula(int n, int k){ int r = 0; for (int i = 2; i <= n; i++) r = (r + k) % i;
return r; }
```

Tests: `tests/test_estructuras.cpp` (todo contra fuerza bruta).

Strings

En el regional casi nunca piden un algoritmo de strings "puro": suele ir mezclado con hashing, conteo o rangos. Para manipulación simple con N ≤ 10^4, Python (`01-sintaxis/python.md`) es lo más rápido de escribir.

Necesito…	Archivo
Comparar subcadenas en O(1), hashear un estado	`rolling_hash.cpp`
Todas las apariciones de un patrón	`kmp.cpp`
Prefijo común entre s y cada sufijo	`z_function.cpp`
Contar subcadenas con paréntesis balanceados	`parentesis_balanceados.cpp`
Rachas de caracteres iguales, colapsar repetidos	`rachas.cpp`, `rachas.py`

[S1] Rolling hash (mod 2^61−1, base aleatoria) — `get(l, r)` = hash de s[l..r) con r EXCLUSIVO. Base aleatoria = resistente a casos anti-hash. Trampa: si dos hashes coinciden, las subcadenas son iguales con probabilidad altísima, no 100 %.

```
// Rolling hash (mod 2^61-1, base aleatoria)
typedef unsigned long long ull;
const ull MOD = (1ULL << 61) - 1;
ull mulmod(ull a, ull b){ __uint128_t c = (__uint128_t)a * b; ull r = (ull)(c & MOD) + (ull)
(c >> 61); return r >= MOD ? r - MOD : r; }
mt19937_64 rng64(chrono::steady_clock::now().time_since_epoch().count());
const ull BASE = rng64() % (MOD - 1000) + 500;
struct Hash {
    vector<ull> h, pw;
    Hash(const string& s) : h(s.size() + 1, 0), pw(s.size() + 1, 1) {
        for (size_t i = 0; i < s.size(); i++){
            h[i+1] = (mulmod(h[i], BASE) + (ull)s[i]) % MOD; pw[i+1] = mulmod(pw[i], BASE); }
}
    ull get(int l, int r){ // hash de s[l..r) (r exclusivo)
        return (h[r] + MOD - mulmod(h[l], pw[r - l])) % MOD; }
};
```

Hash de un vector de frecuencias que cambia: guarda H = Σ freq[i]·BASE^i; al hacer `freq[i]++` suma BASE^i a H (O(1)). Restar 1 a todas las entradas = restar Σ BASE^i (precálculalo). Guarda cada H en un `map<ull,int>` con su primer índice.

[S2] KMP: función prefijo + buscar patrón — O(|txt| + |pat|). `pi[i]` = largo del borde más largo de s[0..i]. Trampa: el separador `#` no debe aparecer en el texto.

```
// KMP (funcion prefijo) + buscar patron
vector<int> prefixFn(const string& s){ int n = s.size(); vector<int> pi(n, 0);
    for (int i = 1; i < n; i++){ int j = pi[i-1]; while (j > 0 && s[i] != s[j]) j = pi[j-1];
if (s[i] == s[j]) j++; pi[i] = j; }
    return pi; }
vector<int> findAll(const string& txt, const string& pat){ // posiciones donde aparece pat
    string s = pat + "#" + txt; auto pi = prefixFn(s); vector<int> res; int m = pat.size();
    for (int i = m + 1; i < (int)s.size(); i++) if (pi[i] == m) res.push_back(i - 2 * m);
    return res; }
```

[S3] Z-function — `z[i]` = largo del mayor prefijo común entre s y s[i..]. `z[0]` queda en 0.

```
// Z-function: z[i] = largo del mayor prefijo comun entre s y s[i..]
vector<int> zfn(const string& s){ int n = s.size(); vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; i++){ if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++; if (i + z[i] > r){ l = i; r =
i + z[i]; } }
    return z; }
```

[S4] Contar subcadenas con paréntesis balanceados — O(n log n). Idea: [l, r) es válida si el balance prefijo en l y r es igual y nunca baja de ese nivel entre medio; al bajar de nivel se "olvidan" los inicios de ese nivel.

```cpp
// Contar subcadenas con parentesis balanceados (prefijo + pila de conteos)
ll countBalanced(const string& s){ // '(' = +1, ')' = -1
    // c[b] = # prefijos con balance b que aun pueden iniciar una subcadena valida
    ll res = 0; int bal = 0; map<int,ll> c; c[0] = 1;
    for (char ch : s){
        if (ch == '('){ bal++; c[bal]++; }
        else { c[bal] = 0; bal--; res += c[bal]; c[bal]++; } }
    return res; }
```

[S5] Reducción canónica (parsing sin gramática) — Cuándo: validar o contar expresiones. Pasos: (1) reemplaza cada nombre de variable por `a` y cada operador binario por `+`; (2) la expresión es válida si los paréntesis están balanceados, no empieza ni termina con `+` y no contiene ninguno de los pares prohibidos `() (+ +) ++ )( )a a(`; (3) corta la cadena en los pares prohibidos y cuenta en cada trozo con [S4] adaptado.

```python
import re
def canon(s):                                   # Python: ideal para este paso
    s = re.sub(r"[A-Za-z_][A-Za-z_0-9]*", "a", s)
    return re.sub(r"[-*/+]", "+", s)
PROHIBIDOS = ["()", "(+", "+)", "++", ")(", ")a", "a("]
```

[S6] Rachas (run-length encoding) — Cuándo: "no puede haber K iguales seguidos", "el mensaje no repite símbolos consecutivos", comprimir. `runs("aaabcc")` da `[(a,3),(b,1),(c,2)]`; `collapse` deja un carácter por racha. En Python sale directo con `itertools.groupby`. Lo usan LAR 2024 K (Kool Strings) y UCAB 2025 B (Borrar).

```cpp
// Rachas (run-length encoding): bloques de caracteres iguales consecutivos
// "aaabcc" -> {('a',3), ('b',1), ('c',2)}
vector<pair<char,int>> runs(const string& s){
    vector<pair<char,int>> r;
    for (char c : s){ if (!r.empty() && r.back().first == c) r.back().second++; else
r.push_back({c, 1}); }
    return r; }
string collapse(const string& s){ string r; for (char c : s) if (r.empty() || r.back() != c)
r += c; return r; } // "aaabcc" -> "abc"
```

```python
from itertools import groupby

def runs(s):
    """'aaabcc' -> [('a', 3), ('b', 1), ('c', 2)]"""
    return [(c, len(list(g))) for c, g in groupby(s)]

def collapse(s):
    """'aaabcc' -> 'abc' (quita repetidos CONSECUTIVOS)"""
    return "".join(c for c, _ in groupby(s))

if __name__ == "__main__":
    assert runs("aaabcc") == [("a", 3), ("b", 1), ("c", 2)]
    assert collapse("ddddaata") == "data"
    print("rachas.py OK")
```

Tests: `tests/test_strings.cpp`.

Matemática

Separada por subtema. Varias plantillas dependen de otras: cada archivo lo dice en su línea `// Requiere:`.

Carpeta	Contenido
`reales/`	`long double`, EPS, búsqueda binaria y ternaria en reales, floor/ceil con negativos
`aritmetica-modular/`	Potencia modular, factoriales y C(n,k) mod p, `__int128` para módulos de 10^18
`teoria-de-numeros/`	Criba, divisores, bases, Euclides extendido, CRT, phi, Miller-Rabin, matrices, sumas
`combinatoria/`	C(n,k) con n enorme, Lucas, Pascal, inclusión-exclusión, desarreglos, inversiones
`probabilidad/`	Valor esperado, fracciones mod p, DP de probabilidades
`permutaciones/`	Componer, invertir, ciclos, potencia
`juegos/`	Grundy y mex

Regla de tipo numérico (decídelo antes de escribir):

Situación	Usa
Todo es entero y cabe en 9·10^18	`ll`
Un producto de dos números ≤ 10^18	`__int128` (`mulmod_int128.cpp`) o Python
Hay π, raíces, senos o divisiones no exactas	`long double` (`base_flotante.cpp`)
Se puede comparar con enteros (a/b < c/d ⇔ a·d < c·b si b, d > 0)	Enteros, siempre
Enteros gigantes o fracciones exactas, pocas operaciones	Python (`int`, `Fraction`)

Reales (flotantes)

[F1] Base flotante — Cópialo arriba de cualquier problema con decimales. `long double` da ~18 cifras (`double` ~15). Trampa: nunca compares flotantes con `==`; usa `sgn`/`eq`.

```cpp
// Base flotante
typedef long double ld;
const ld PI = acosl(-1);
const ld EPS = 1e-9;
int sgn(ld x){ return (x > EPS) - (x < -EPS); }      // -1, 0, 1 con tolerancia
bool eq(ld a, ld b){ return fabsl(a - b) < EPS; }
```

Con `long double` usa las versiones con `l`: `sqrtl`, `fabsl`, `sinl`, `cosl`, `tanl`, `atan2l`, `acosl`, `hypotl`, `floorl`, `powl`. Leer: `ld x; cin >> x;` funciona igual.

[F2] Imprimir decimales — Cuando dicen "error absoluto o relativo ≤ 10^-6", imprime 9 cifras o más.

```cpp
cout << fixed << setprecision(9) << x << "\n";   // ponlo una vez al inicio de main
printf("%.9Lf\n", x);                            // ld con printf: %Lf (L mayuscula)
```

```python
print(f"{x:.9f}")
```

Trampa: si la salida esperada es entera ("16", "300") y el juez acepta error relativo, imprimir `16.000000000` también es correcto. Si piden un entero exacto, usa `llround(x)`.

[F3] Búsqueda binaria en reales — Cuándo: buscar x con f(x) = objetivo y f creciente (alturas, tiempos, radios). 200 iteraciones siempre alcanzan; no uses `while (hi - lo > EPS)` (puede no terminar).

```cpp
// Busqueda binaria en reales: f creciente, buscar x con f(x) = target
// Requiere: reales/base_flotante.cpp (ld)
template<class F> ld bsReal(ld lo, ld hi, ld target, F f){
    for (int it = 0; it < 200; it++){ ld mid = (lo + hi) / 2; if (f(mid) < target) lo = mid;
else hi = mid; }
    return (lo + hi) / 2; }
```

[F4] Búsqueda ternaria — Cuándo: f sube y luego baja (unimodal) y buscas el máximo. Para mínimo, cambia < por >. Trampa: con zonas planas en el medio falla.

```cpp
// Busqueda ternaria: maximo de f unimodal en [lo, hi]
// Requiere: reales/base_flotante.cpp (ld)
template<class F> ld ternaryMax(ld lo, ld hi, F f){
    for (int it = 0; it < 200; it++){ ld m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) lo = m1; else hi = m2; }
    return (lo + hi) / 2; }
```

[F5] Funciones seguras — Por redondeo, `sqrt(-1e-18)` da NaN y `acos(1.0000001)` también. Usa siempre estas versiones.

```cpp
// Funciones seguras
// Requiere: reales/base_flotante.cpp (ld)
ld safeSqrt(ld x){ return sqrtl(max((ld)0, x)); }
ld safeAcos(ld x){ return acosl(max((ld)-1, min((ld)1, x))); }
```

[F6] División entera hacia abajo/arriba con negativos — En C++ `-7 / 2 == -3` (trunca hacia 0), no −4.

```cpp
// Division entera con negativos (floor/ceil reales)
ll floorDiv(ll a, ll b){ return a / b - ((a % b != 0) && ((a < 0) != (b < 0))); }
ll ceilDiv(ll a, ll b){ return a / b + ((a % b != 0) && ((a < 0) == (b < 0))); }
```

[F7] Python para flotantes y exactitud

```python
import math
from fractions import Fraction
from decimal import Decimal, getcontext
math.pi; math.radians(180); math.degrees(math.pi); math.atan2(y, x); math.hypot(3, 4)
math.isclose(0.1 + 0.2, 0.3)        # True (comparar flotantes)
math.floor(-2.5), math.ceil(-2.5)   # -3, -2
-7 // 2                             # -4 (floor: redondea hacia -inf)
-(-a // b)                          # ceil(a / b) con enteros: -(-7 // 2) = 4
f = Fraction(1, 6) + Fraction(1, 3) # 1/2 exacto; f.numerator, f.denominator
getcontext().prec = 50; Decimal(2).sqrt()   # 50 cifras
```

Trampa: `round(2.675, 2)` da `2.67` (2.675 no es exacto en binario). Para imprimir usa f-strings y deja que el juez tolere el error.

Aritmética modular

[M1] Aritmética modular y combinatoria — Cuándo: "imprime la respuesta módulo 10^9+7". Trampa: reduce con `% MD` después de CADA producto; `inv` solo sirve si MD es primo. Llama `initFact()` una vez en `main`.

```cpp
// Modular: potencia, inverso, combinatoria
// Requiere: const ll MD (esta en la plantilla base)
ll pw(ll b, ll e, ll m = MD){ ll r = 1; b %= m; if (b < 0) b += m; while (e){ if (e & 1) r = r * b % m; b = b * b % m; e >>= 1; } return r; }
ll inv(ll a){ return pw(a, MD - 2); } // MD primo
const int MAXF = 1000005; ll fact[MAXF], ifact[MAXF];
void initFact(){ fact[0] = 1; for (int i = 1; i < MAXF; i++) fact[i] = fact[i-1] * i % MD;
    ifact[MAXF-1] = inv(fact[MAXF-1]); for (int i = MAXF - 1; i > 0; i--) ifact[i-1] = ifact[i] * i % MD; }
ll C(int n, int k){ if (k < 0 || k > n) return 0; return fact[n] * ifact[k] % MD * ifact[n-k] % MD; }
```

Fórmulas útiles: estrellas y barras (repartir n iguales en k cajas) = C(n+k−1, k−1); caminos en grilla a×b = C(a+b, a); Catalan(n) = C(2n, n)/(n+1).

[CB1] Multiplicar con módulos de hasta ~4·10^18 — Cuándo: el módulo es como 10^18+3 (UCAB 2025 K: respuesta k² mod 10^18+3). Trampa: `a * b % m` con `ll` desborda si a·b > 9·10^18.

```cpp
// Multiplicacion y potencia con modulos grandes (hasta ~4e18)
typedef __int128 lll;
ll mulmod(ll a, ll b, ll m){ return (ll)((lll)a * b % m); }
ll powmod(ll b, ll e, ll m){ ll r = 1 % m; b %= m; if (b < 0) b += m;
    while (e){ if (e & 1) r = mulmod(r, b, m); b = mulmod(b, b, m); e >>= 1; } return r; }
void print128(lll x){ if (x < 0){ putchar('-'); x = -x; } if (x > 9) print128(x / 10);
putchar('0' + (int)(x % 10)); }
```

`cout` no imprime `__int128`: usa `print128` (con `printf`/`putchar`, no mezcles con `cout` sin `cout.flush()`). En Python: `k * k % (10**18 + 3)`.

Teoría de números

[M2] Criba, divisores, factorización — `spf[x]` = menor primo que divide x (factoriza cualquier x ≤ n en O(log x)). `divisors` y `factorize` son O(√n).

```cpp
// Criba, divisores, factorizacion
vector<int> sieve(int n){ vector<int> spf(n + 1, 0); // spf[x] = menor primo que divide x
    for (int i = 2; i <= n; i++) if (!spf[i]) for (int j = i; j <= n; j += i) if (!spf[j]) spf[j] = i;
    return spf; }
vector<ll> divisors(ll n){ vector<ll> d; for (ll i = 1; i * i <= n; i++) if (n % i == 0){ d.push_back(i); if (i != n / i) d.push_back(n / i); }
    sort(d.begin(), d.end()); return d; }
vector<pair<ll,int>> factorize(ll n){ vector<pair<ll,int>> f;
    for (ll p = 2; p * p <= n; p++) if (n % p == 0){ int e = 0; while (n % p == 0){ n /= p; e++; } f.push_back({p, e}); }
    if (n > 1) f.push_back({n, 1}); return f; }
```

[M3] Bases numéricas y palíndromos — Cuándo: "¿en qué bases b es N palíndromo?". Truco O(√N): para b ≤ √N prueba con fuerza bruta; para b > √N el número tiene 2 dígitos `dd` y N = d·(b+1), así que b = N/d − 1 para cada divisor d de N con d < b.

```cpp
// Base b: digitos y palindromo
vector<ll> toBase(ll n, ll b){ vector<ll> d; if (n == 0) d.push_back(0); while (n){ d.push_back(n % b); n /= b; } reverse(d.begin(), d.end()); return d; }
bool isPal(const vector<ll>& d){ for (size_t i = 0, j = d.size() - 1; i < j; i++, j--) if (d[i] != d[j]) return false; return true; }
```

[NT1] Euclides extendido — Encuentra x, y con a·x + b·y = gcd(a, b).

```cpp
// Euclides extendido: a*x + b*y = g = gcd(a, b)
ll extgcd(ll a, ll b, ll &x, ll &y){ if (b == 0){ x = 1; y = 0; return a; }
    ll x1, y1, g = extgcd(b, a % b, x1, y1); x = y1; y = x1 - (a / b) * y1; return g; }
```

[NT2] Ecuación diofántica a·x + b·y = c — Tiene solución si y solo si gcd(a, b) divide a c. Todas las soluciones: (x + k·b/g, y − k·a/g) para k entero; elige k para que x, y ≥ 0 si lo piden.

```cpp
// Diofantica a*x + b*y = c (a, b > 0): una solucion; todas = (x + k*b/g, y - k*a/g)
// Requiere: teoria-de-numeros/extgcd.cpp, aritmetica-modular/mulmod_int128.cpp (lll)
bool diophantine(ll a, ll b, ll c, ll &x, ll &y){ ll g = extgcd(a, b, x, y); if (c % g)
return false;
    x = (ll)((lll)x * (c / g)); y = (ll)((lll)y * (c / g)); return true; }
```

[NT3] Inverso modular con módulo NO primo — Existe solo si gcd(a, m) = 1. Si m es primo, usa `powmod(a, m - 2, m)`.

```cpp
// Inverso modular con modulo NO primo (existe si gcd(a, m) = 1), -1 si no existe
// Requiere: teoria-de-numeros/extgcd.cpp
ll invGeneral(ll a, ll m){ ll x, y; ll g = extgcd(((a % m) + m) % m, m, x, y); if (g != 1)
return -1; return ((x % m) + m) % m; }
```

[NT4] Teorema chino del resto — Cuándo: "x deja resto r1 al dividir entre m1 y resto r2 entre m2". Sirve aunque m1, m2 no sean coprimos. Para más ecuaciones, combínalas de a dos.

```cpp
// Teorema chino del resto (modulos no necesariamente coprimos)
// Requiere: teoria-de-numeros/extgcd.cpp, aritmetica-modular/mulmod_int128.cpp (lll)
// x = r1 (mod m1), x = r2 (mod m2) -> {r, lcm} o {-1, -1} si no hay solucion
pair<ll,ll> crt(ll r1, ll m1, ll r2, ll m2){ ll p, q; ll g = extgcd(m1, m2, p, q);
    if ((r2 - r1) % g) return {-1, -1};
    ll l = m1 / g * m2; lll t = (lll)(r2 - r1) / g * p % (m2 / g);
    ll r = (ll)(((lll)r1 + (lll)m1 * t) % l); if (r < 0) r += l; return {r, l}; }
```

[NT5] Phi de Euler — phi(n) = cuántos de 1..n son coprimos con n. Propiedad: a^phi(m) ≡ 1 (mod m) si gcd(a, m) = 1, así que a^e mod m = a^(e mod phi(m)) mod m.

```cpp
// Phi de Euler: phi(n) = cantidad de 1..n coprimos con n
ll phi(ll n){ ll r = n; for (ll p = 2; p * p <= n; p++) if (n % p == 0){ while (n % p == 0) n /= p; r -= r / p; } if (n > 1) r -= r / n; return r; }
vector<int> phiSieve(int n){ vector<int> ph(n + 1); iota(ph.begin(), ph.end(), 0);
    for (int i = 2; i <= n; i++) if (ph[i] == i) for (int j = i; j <= n; j += i) ph[j] -= ph[j] / i; return ph; }
```

[NT6] Cantidad y suma de divisores en O(√n) — Cuando n = p1^e1·…·pk^ek: d(n) = (e1+1)·…·(ek+1), sigma(n) = Π (1 + p + … + p^e). Suma de divisores propios = sigma(n) − n (perfecto / abundante / deficiente).

```cpp
// Numero y suma de divisores desde la factorizacion n = p1^e1 * ... * pk^ek
// d(n) = (e1+1)(e2+1)...   sigma(n) = prod (p^(e+1) - 1) / (p - 1)
pair<ll,ll> divCountSum(ll n){ ll cnt = 1, sum = 1;
    for (ll p = 2; p * p <= n; p++) if (n % p == 0){ int e = 0; ll pk = 1, s = 1; while (n % p == 0){ n /= p; e++; pk *= p; s += pk; } cnt *= e + 1; sum *= s; }
    if (n > 1){ cnt *= 2; sum *= n + 1; } return {cnt, sum}; }
```

[NT7] ¿Es primo? (n hasta 2^64) — Miller-Rabin determinista, O(log³ n). Usa para n > 10^12, donde la división hasta √n es lenta.

```cpp
// Miller-Rabin determinista para n < 2^64
// Requiere: aritmetica-modular/mulmod_int128.cpp
bool isPrime(ll n){ if (n < 2) return false;
    for (ll p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) if (n % p == 0) return n == p;
    ll d = n - 1; int s = 0; while (d % 2 == 0){ d /= 2; s++; }
    for (ll a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}){ ll x = powmod(a, d, n); if (x
== 1 || x == n - 1) continue;
        bool comp = true; for (int r = 1; r < s; r++){ x = mulmod(x, x, n); if (x == n - 1){
comp = false; break; } }
        if (comp) return false; }
    return true; }
```

[NT8] Exponenciación de matrices — Cuándo: recurrencia lineal f(n) = c1·f(n−1) + ... con n hasta 10^18. O(k³ log n) con matriz k×k. Trampa: con módulo > 3·10^9 el producto `A * B` desborda: cambia por `mulmod`.

```cpp
// Exponenciacion de matrices (recurrencias lineales: Fibonacci, etc.)
typedef vector<vector<ll>> Mat;
Mat mul(const Mat& A, const Mat& B, ll m){ int n = A.size(), k = B.size(), p = B[0].size();
Mat C(n, vector<ll>(p, 0));
    for (int i = 0; i < n; i++) for (int l = 0; l < k; l++) if (A[i][l]) for (int j = 0; j <
p; j++) C[i][j] = (C[i][j] + A[i][l] * B[l][j]) % m;
    return C; }
Mat matpow(Mat A, ll e, ll m){ int n = A.size(); Mat R(n, vector<ll>(n, 0)); for (int i = 0;
i < n; i++) R[i][i] = 1 % m;
    while (e){ if (e & 1) R = mul(R, A, m); A = mul(A, A, m); e >>= 1; } return R; }
ll fib(ll n, ll m){ if (n == 0) return 0; Mat F = {{1, 1}, {1, 0}}; return matpow(F, n - 1,
m)[0][0]; } // F(1)=F(2)=1
```

[NT9] Sumas cerradas — Con `__int128` para no desbordar con n hasta 10^9 o más. `geoSum` sirve con cualquier módulo (no divide).

```cpp
// Sumas cerradas (con __int128 para no desbordar)
// Requiere: aritmetica-modular/mulmod_int128.cpp
lll sum1(ll n){ return (lll)n * (n + 1) / 2; }                          // 1 + 2 + ... + n
lll sum2(ll n){ return (lll)n * (n + 1) * (2 * n + 1) / 6; }            // 1^2 + ... + n^2
lll sum3(ll n){ lll s = sum1(n); return s * s; }                         // 1^3 + ... + n^3
ll geoSum(ll r, ll n, ll m){ // 1 + r + r^2 + ... + r^(n-1) mod m, m cualquiera
    if (n == 0) return 0; if (n % 2 == 1) return (1 + mulmod(r % m, geoSum(r, n - 1, m), m))
% m;
    ll h = geoSum(r, n / 2, m); return mulmod(h, (1 + powmod(r, n / 2, m)) % m, m); }
```

Suma de a hasta b = sum1(b) − sum1(a − 1). Progresión aritmética: n·(primero + último)/2.

[NT10] Bloques de n / i — Cuándo: Σ f(n / i) para i = 1..n con n hasta 10^12. n / i toma solo ~2√n valores distintos; cada bloque [i, j] comparte el mismo cociente q.

```cpp
// Bloques de n / i: suma de floor(n / i) para i = 1..n en O(sqrt n)
ll sumFloor(ll n){ ll s = 0; for (ll i = 1; i <= n; ){ ll q = n / i, j = n / q; s += q * (j -
i + 1); i = j + 1; } return s; }
```

Combinatoria

[CB2] C(n, k) con n hasta 10^18 y k pequeño — Cuándo: "cuántos triángulos con vértices de un polígono de n lados" (UCAB 2025 I: C(n, 3) mod 10^9+7). O(k log p).

```cpp
// C(n, k) mod p primo con n enorme (hasta 1e18) y k pequeno (k < p)
// Requiere: aritmetica-modular/mulmod_int128.cpp
ll nCrBig(ll n, ll k, ll p){ if (k < 0 || k > n) return 0; ll num = 1, den = 1;
    for (ll i = 0; i < k; i++){ num = mulmod(num, (n - i) % p, p); den = mulmod(den, (i + 1)
% p, p); }
    return mulmod(num, powmod(den, p - 2, p), p); }
```

En Python sale directo con enteros grandes: `n * (n - 1) * (n - 2) // 6 % MOD` (divide ANTES del módulo).

[CB3] Lucas (p primo pequeño)

```cpp
// Lucas: C(n, k) mod p primo PEQUENO (p <= ~1e6), n y k enormes
// Requiere: aritmetica-modular/mulmod_int128.cpp
ll nCrSmall(ll n, ll k, ll p){ if (k < 0 || k > n) return 0; ll r = 1;
    for (ll i = 0; i < k; i++){ r = r * ((n - i) % p) % p; r = r * powmod(i + 1, p - 2, p) %
p; } return r; }
ll lucas(ll n, ll k, ll p){ if (k == 0) return 1; return nCrSmall(n % p, k % p, p) * lucas(n
/ p, k / p, p) % p; }
```

[CB4] Triángulo de Pascal (cualquier módulo) — O(n²) memoria y tiempo.

```cpp
// Triangulo de Pascal: C(n, k) mod CUALQUIER m (n <= ~5000)
vector<vector<ll>> pascal(int N, ll m){ vector<vector<ll>> C(N + 1, vector<ll>(N + 1, 0));
    for (int n = 0; n <= N; n++){ C[n][0] = 1 % m; for (int k = 1; k <= n; k++) C[n][k] =
(C[n-1][k-1] + C[n-1][k]) % m; }
    return C; }
```

[CB5] Inclusión-exclusión — Cuándo: "cuántos de 1..N cumplen al menos una condición". Suma las intersecciones de tamaño impar y resta las de tamaño par. Ejemplo: divisibles por alguno de a[] (k ≤ 20).

```cpp
// Inclusion-exclusion: cuantos x en [1, N] son divisibles por AL MENOS uno de a[]
// Requiere: aritmetica-modular/mulmod_int128.cpp (lll)
ll countDivisible(ll N, vector<ll>& a){ int k = a.size(); ll res = 0;
    for (int mask = 1; mask < (1 << k); mask++){ ll l = 1; bool big = false;
        for (int i = 0; i < k; i++) if (mask >> i & 1){ l = l / gcd(l, a[i]); if ((lll)l *
a[i] > N){ big = true; break; } l *= a[i]; }
        if (big) continue; ll c = N / l; res += (__builtin_popcount(mask) & 1) ? c : -c; }
    return res; }
```

[CB6] Desarreglos y Stirling

```cpp
// Desarreglos (nadie en su lugar) y Stirling 2da especie
vector<ll> derangements(int n, ll m){ vector<ll> D(max(n + 1, 2), 0); D[0] = 1 % m; D[1] = 0;
    for (int i = 2; i <= n; i++) D[i] = (ll)(i - 1) * ((D[i-1] + D[i-2]) % m) % m; return D;
}
// S[n][k] = formas de repartir n elementos distintos en k grupos no vacios
vector<vector<ll>> stirling2(int N, ll m){ vector<vector<ll>> S(N + 1, vector<ll>(N + 1, 0));
S[0][0] = 1;
    for (int n = 1; n <= N; n++) for (int k = 1; k <= n; k++) S[n][k] = (S[n-1][k-1] + (ll)k
* S[n-1][k]) % m; return S; }
```

Contar…	Fórmula
Subconjuntos de n	2^n
Ordenar n con repetidos (c_1, c_2, …)	n! / (c_1!·c_2!·…)
Funciones de k en n	n^k; sobreyectivas (todo n usado): n!·S[k][n]
Pares no ordenados de n	n(n−1)/2
Desarreglos D(n)	0, 1, 2, 9, 44, 265 para n = 1..6

[M4] Permutaciones con x inversiones (DP + sumas prefijas) — f[n][x] = Σ f[n−1][x−j] para 0 ≤ j < n, optimizado a O(n·X). `cap` evita desborde cuando K llega a 10^18.

```cpp
// f[n][x] = # permutaciones de n con x inversiones (sumas prefijas, O(n*X))
vector<vector<ll>> invCount(int N, int X, ll cap = LLONG_MAX / 4){ // cap: tope para no
desbordar
    vector<vector<ll>> f(N + 1, vector<ll>(X + 1, 0)); f[0][0] = 1;
    for (int n = 1; n <= N; n++){ vector<ll> g(X + 1); // g = prefijo de f[n-1]
        for (int x = 0; x <= X; x++) g[x] = min(cap, (x ? g[x-1] : 0) + f[n-1][x]);
        for (int x = 0; x <= X; x++) f[n][x] = min(cap, g[x] - (x - n >= 0 ? g[x-n] : 0)); }
    return f; }
```

K-ésima permutación con x inversiones: con r elementos restantes, elegir el j-ésimo menor disponible (j = 0..r−1) aporta j inversiones y deja f[r−1][x−j] completaciones. Recorre j: si K > ese número, réstalo de K; si no, toma ese elemento (extráelo con [D1] `kth` o [D2]) y resta j a x.

Probabilidad

Reglas que resuelven casi todo

Regla	Uso
Linealidad: E[X + Y] = E[X] + E[Y] (aunque dependan)	n tareas iguales → n · E[una tarea]

Regla	Uso
Geométrica: si cada intento sale bien con prob p, E[intentos] = 1/p	"tirar hasta que salga"
Con repetición: E = p_fin·c_fin + p_rep·(c_rep + E) → despejar E	Ver [PR1]
Dos dados de 6 caras	P(iguales) = 1/6, P(A > B) = P(B > A) = 5/12
Independientes: P(A y B) = P(A)·P(B)	
Complemento: P(al menos uno) = 1 − P(ninguno)	

[PR1] Valor esperado con "repetir" — En cada ronda, con prob `pRep` se repite gastando `cRep`; si no, termina gastando en promedio `costFinal`Promedio. Despejando E = p_fin·c_fin + p_rep·(c_rep + E). Probado con los 4 ejemplos de UCAB 2025 A.

```
// Requiere: reales/base_flotante.cpp (ld)
// Valor esperado con "repetir": E = (sum p_i * costo_i) / (1 - p_repetir)
// en cada intento: con prob pRep se repite gastando cRep; si no, termina gastando E_fin
ld expectedRetry(ld pRep, ld cRep, ld costFinalPromedio){ return (costFinalPromedio * (1 - pRep) + pRep * cRep) / (1 - pRep); }
```

```
// Base flotante
typedef long double ld;
const ld PI = acosl(-1);
const ld EPS = 1e-9;
int sgn(ld x){ return (x > EPS) - (x < -EPS); }      // -1, 0, 1 con tolerancia
bool eq(ld a, ld b){ return fabsl(a - b) < EPS; }
```

[PR2] Probabilidad como fracción módulo p — Cuándo: "imprime la probabilidad como P·Q⁻¹ mod 10^9+7". Suma y multiplica fracciones directamente en mod.

```
const ll MD = 1e9 + 7;
ll fracMod(ll P, ll Q){ return mulmod(P % MD, powmod(Q, MD - 2, MD), MD); }   // usa [CB1]
// 1/2 -> 500000004
```

[PR3] DP de probabilidad — Cuándo: distribución de una suma o de un proceso por pasos. `dp[estado]` = probabilidad de estar ahí; cada paso reparte la probabilidad. Ejemplo: suma de n dados de c caras.

```
// DP de probabilidad: distribucion de la suma de n dados de c caras
// Requiere: reales/base_flotante.cpp (ld)
vector<ld> diceSum(int n, int c){ vector<ld> dp(1, 1.0L);
    for (int t = 0; t < n; t++){ vector<ld> nd(dp.size() + c, 0);
        for (size_t s = 0; s < dp.size(); s++) if (dp[s] > 0) for (int f = 1; f <= c; f++) nd[s + f] += dp[s] / c;
        dp = nd; }
    return dp; } // dp[s] = P(suma = s)
```

Valor esperado por DP: E[estado] = Σ P(transición)·(costo + E[siguiente]). Si un estado puede volver a sí mismo, despeja como en [PR1].

Permutaciones

[PE1] Componer, invertir, ciclos, potencia — Cuándo: intercambios de filas/columnas, "aplicar la mezcla k veces", "cuántas veces hasta volver al inicio".

```
// Permutaciones 0-indexadas: componer, inversa, ciclos
vector<int> compose(const vector<int>& p, const vector<int>& q){ // (p o q)[i] = p[q[i]]
    int n = p.size(); vector<int> r(n); for (int i = 0; i < n; i++) r[i] = p[q[i]]; return r;
}
vector<int> inverse(const vector<int>& p){ int n = p.size(); vector<int> r(n); for (int i = 0; i < n; i++) r[p[i]] = i; return r; }
vector<vector<int>> cycles(const vector<int>& p){ int n = p.size(); vector<bool> vis(n);
vector<vector<int>> cs;
    for (int i = 0; i < n; i++) if (!vis[i]){ vector<int> c; for (int j = i; !vis[j]; j = p[j]){ vis[j] = true; c.push_back(j); } cs.push_back(c); }
    return cs; }
// swaps minimos para ordenar = n - #ciclos; paridad = (n - #ciclos) % 2; orden (veces hasta volver a identidad) = lcm de largos
vector<int> permPow(const vector<int>& p, ll k){ // p aplicada k veces, O(n)
    vector<int> r(p.size());
    for (auto &c : cycles(p)){ int L = c.size(); for (int i = 0; i < L; i++) r[c[i]] = c[(i + k % L) % L]; }
    return r; }
```

Pregunta	Respuesta con los ciclos
Mínimo de intercambios (swaps) para ordenar	n − (número de ciclos)
Paridad (par/impar)	(n − ciclos) % 2; cada swap cambia la paridad
Veces hasta volver a la identidad (orden)	lcm de los largos de los ciclos (puede ser enorme: usa mod o __int128)
¿p^k = identidad?	k es múltiplo del largo de cada ciclo

Intercambios sucesivos de filas (o columnas): lleva un arreglo `row[i]` = fila original que está ahora en la posición i, aplica cada `swap(row[I], row[J])` en orden, y al final `row` es la permutación total. Filas y columnas son independientes.

Juegos y constructivos

[M5] Juegos: Grundy y mex — Juego imparcial: grundy(estado) = mex de los grundy de los estados alcanzables; varios juegos independientes → XOR de sus grundy; el primero pierde si el XOR es 0. Antes de programar Grundy, prueba casos pequeños a mano y busca un invariante (paridad, módulo 3, simetría/imitar al rival).

```
// Grundy / mex
int mex(vector<int>& v){ set<int> s(v.begin(), v.end()); int m = 0; while (s.count(m)) m++;
return m; }
```

[M6] Patrones constructivos — Cuándo: "construye una secuencia/arreglo que cumpla...".

Patrón	Idea
Pensar hacia atrás	Simula desde el estado final al inicial (pliegues, eliminaciones)
Extremos + inducción	Demuestra dónde va el 1 (o el máximo), quítalo y repite con n−1; dos punteros `l`, `r`
Sistema lineal de bloques	Si la solución se arma con bloques de tamaño 2 y 3: n = 2·a + 3·b y k = a + b → a = 3k − n, b = n − 2k (verifica a, b ≥ 0)
Fuerza bruta pequeña	Genera todas las respuestas para n ≤ 8 en Python y busca el patrón

Tests: `tests/test_matematica.cpp` (contra fuerza bruta y con los ejemplos de UCAB 2025 A, I, K).

Geometría

Dos mundos: si todas las coordenadas son enteras y solo necesitas cruz, punto o área, trabaja **en enteros** (exacto). Si aparecen raíces, círculos, ángulos o divisiones, usa **flotantes** con `long double`.

Carpeta	Contenido
`enteros/`	Punto entero, orientación, envolvente convexa (con y sin colineales), intersección de segmentos, orden angular
`flotantes/`	Punto real, proyecciones, distancias, rectas, círculos, polígonos
`trigonometria/`	Grados/radianes, atan2, leyes de senos y cosenos, Herón
`volumenes/`	Conos, esferas, cilindros, casquetes; altura de llenado

Enteros

[GE1] Punto, cruz, orientación, área — `orient(a,b,c)` > 0 si c está a la IZQUIERDA de a→b (giro antihorario), < 0 a la derecha, 0 colineal. `area2` devuelve 2·área con signo (positiva si el polígono va antihorario).

```
// Geometria en enteros
struct P { ll x, y;
    P operator-(const P& o) const { return {x - o.x, y - o.y}; }
    P operator+(const P& o) const { return {x + o.x, y + o.y}; }
    bool operator<(const P& o) const { return x != o.x ? x < o.x : y < o.y; }
    bool operator==(const P& o) const { return x == o.x && y == o.y; } };
ll cross(P a, P b){ return a.x * b.y - a.y * b.x; }
ll dot(P a, P b){ return a.x * b.x + a.y * b.y; }
ll orient(P a, P b, P c){ return cross(b - a, c - a); } // >0 izq (antihorario), <0 der, 0 colineal
ll area2(vector<P>& p){ ll s = 0; int n = p.size(); // 2*area con signo (shoelace)
    for (int i = 0; i < n; i++) s += cross(p[i], p[(i + 1) % n]); return s; }
```

[GE2] Envolvente convexa (monotone chain) — O(n log n), antihorario, sin puntos colineales. No la modifiques para incluir colineales: cambiar a `< 0` falla si todos los puntos están en una recta.

```
// Convex hull (monotone chain), antihorario, sin colineales
// Requiere: enteros/punto_entero.cpp
vector<P> hull(vector<P> p){ sort(p.begin(), p.end()); p.erase(unique(p.begin(), p.end()),
p.end());
    int n = p.size(); if (n < 3) return p; vector<P> h(2 * n); int k = 0;
    for (int i = 0; i < n; i++){ while (k >= 2 && orient(h[k-2], h[k-1], p[i]) <= 0) k--;
h[k++] = p[i]; }
    for (int i = n - 2, t = k + 1; i >= 0; i--){ while (k >= t && orient(h[k-2], h[k-1],
p[i]) <= 0) k--; h[k++] = p[i]; }
    h.resize(k - 1); return h; }
```

[GE3] ¿Se cortan dos segmentos? — Incluye tocarse en un extremo y solaparse colineales.

```
// Interseccion de segmentos (incluye extremos)
// Requiere: enteros/punto_entero.cpp
bool onSeg(P a, P b, P p){ return orient(a, b, p) == 0 && min(a.x,b.x) <= p.x && p.x <=
max(a.x,b.x) && min(a.y,b.y) <= p.y && p.y <= max(a.y,b.y); }
int sgn(ll v){ return (v > 0) - (v < 0); }
bool segInter(P a, P b, P c, P d){
    int o1 = sgn(orient(a,b,c)), o2 = sgn(orient(a,b,d)), o3 = sgn(orient(c,d,a)), o4 =
sgn(orient(c,d,b));
    if (o1 != o2 && o3 != o4) return true;
    return onSeg(a,b,c) || onSeg(a,b,d) || onSeg(c,d,a) || onSeg(c,d,b); }
```

[GE4] Orden angular (barrido radial) — Ordena vectores por ángulo desde 0° hasta 360°, sin `atan2` ni flotantes. Para ordenar alrededor de un pivote q, ordena `p[i] - q`.

```
// Orden angular alrededor del origen (barrido radial)
// Requiere: enteros/punto_entero.cpp
int half(P p){ return p.y < 0 || (p.y == 0 && p.x < 0); } // 0: [0,180)  1: [180,360)
bool angLess(P a, P b){ if (half(a) != half(b)) return half(a) < half(b); return cross(a, b)
> 0; }
```

Otras técnicas que aparecen en el regional (sin plantilla aquí): rotating calipers sobre el hull (diámetro, ancho), línea de barrido con eventos ordenados, intersección de semiplanos O(n log n).

[GE5] Envolvente con los puntos colineales del borde — Cuándo: el enunciado cuenta TODOS los puntos que están sobre el borde (por ejemplo, "capas" de una cebolla de puntos, UCV 2025 G si los colineales cuentan). Es el mismo monotone chain con `< 0`, más el caso especial de que todos estén en una recta. Probado contra fuerza bruta.

```
// Convex hull que INCLUYE los puntos colineales del borde (monotone chain).
// Requiere: punto_entero.cpp
// Caso especial: si todos los puntos estan en una recta, devuelve todos ordenados.
vector<P> hullWithCollinear(vector<P> p){
    sort(p.begin(), p.end()); p.erase(unique(p.begin(), p.end()), p.end());
    int n = p.size(); if (n < 3) return p;
    bool allCol = true; for (int i = 2; i < n; i++) if (orient(p[0], p[1], p[i]) != 0) allCol
= false;
    if (allCol) return p;
    vector<P> h(2 * n); int k = 0;
    for (int i = 0; i < n; i++){ while (k >= 2 && orient(h[k-2], h[k-1], p[i]) < 0) k--;
h[k++] = p[i]; }
    for (int i = n - 2, t = k + 1; i >= 0; i--){ while (k >= t && orient(h[k-2], h[k-1],
p[i]) < 0) k--; h[k++] = p[i]; }
    h.resize(k - 1); return h; }
```

Trigonometría

[TR1] Grados, radianes y ángulos — Las funciones de C++ y Python trabajan en RADIANES. Trampa: si el enunciado da grados (θ = 45), convierte antes de `sin`/`cos`/`tan`.

```
// Grados/radianes y angulos
// Requiere: reales/base_flotante.cpp
ld toRad(ld deg){ return deg * PI / 180; }
ld toDeg(ld rad){ return rad * 180 / PI; }
ld angle360(ld y, ld x){ ld a = atan2l(y, x); return a < 0 ? a + 2 * PI : a; } // [0, 2pi)
```

Función	Devuelve	Nota
`atan2l(y, x)`	Ángulo del vector (x, y) en (−π, π]	Usa esta, no `atan(y/x)` (falla con x = 0 y pierde el cuadrante)
`acosl(v)`	[0, π]	Pasa por `safeAcos` [F5]
`asinl(v)`	[−π/2, π/2]	Igual: recorta a [−1, 1]
`tanl(t)`	tan	Explota cerca de 90°

[TR2] Triángulos — Lados a, b, c; el ángulo C es el opuesto al lado c.

```
// Triangulos: lados a,b,c (c opuesto a C)
// Requiere: reales/funciones_seguras.cpp
ld angleFromSides(ld a, ld b, ld c){ return safeAcos((a*a + b*b - c*c) / (2*a*b)); } //
angulo C
ld sideFromAngle(ld a, ld b, ld C){ return safeSqrt(a*a + b*b - 2*a*b*cosl(C)); }   // lado c
ld heron(ld a, ld b, ld c){ ld s = (a + b + c) / 2; return safeSqrt(s * (s-a) * (s-b) * (s-
c)); }
ld areaSAS(ld a, ld b, ld C){ return a * b * sinl(C) / 2; }
// ley de senos: a / sin(A) = b / sin(B) = c / sin(C) = 2R  (R = radio circunscrito)
```

Necesito	Fórmula
Área con base y altura	b·h / 2
Radio circunscrito	R = a·b·c / (4·área)
Radio inscrito	r = área / s, con s = (a+b+c)/2
Altura sobre el lado c	2·área / c
Sombra de un objeto de altura h con luz a θ grados	h / tan(θ)
Triángulo equilátero de lado s	área = √3·s²/4, altura = √3·s/2

[TR3] Rotación, ángulo entre vectores, polares — Usa el struct `PT` de [GF1].

```cpp
// Punto/vector flotante
// Requiere: 06-matematica/reales/base_flotante.cpp
struct PT { ld x, y;
    PT operator+(PT o) const { return {x + o.x, y + o.y}; }
    PT operator-(PT o) const { return {x - o.x, y - o.y}; }
    PT operator*(ld k) const { return {x * k, y * k}; }
    PT operator/(ld k) const { return {x / k, y / k}; } };
ld dot(PT a, PT b){ return a.x * b.x + a.y * b.y; }
ld cross(PT a, PT b){ return a.x * b.y - a.y * b.x; }
ld norm(PT a){ return hypotl(a.x, a.y); }
ld dist(PT a, PT b){ return norm(a - b); }
PT perp(PT a){ return {-a.y, a.x}; }                                  // rota 90 antihorario
PT rotate(PT a, ld t){ return {a.x * cosl(t) - a.y * sinl(t), a.x * sinl(t) + a.y * cosl(t)};
}
ld angleBetween(PT a, PT b){ return fabsl(atan2l(cross(a, b), dot(a, b))); }  // [0, pi]
PT polar(ld r, ld t){ return {r * cosl(t), r * sinl(t)}; }
```

Rotar alrededor de un punto c: `rotate(p - c, t) + c`. Rotar 90° exacto sin flotantes: (x, y) → (−y, x).

Geometría con flotantes

Si todas las coordenadas son enteras y solo necesitas cruz/punto/área, usa la versión entera del primer cheatsheet ([GE]). Esta es para cuando aparecen raíces, divisiones o círculos.

[GF1] Punto / vector — Base de todo lo demás; requiere [F1].

```cpp
// Punto/vector flotante
// Requiere: 06-matematica/reales/base_flotante.cpp
struct PT { ld x, y;
    PT operator+(PT o) const { return {x + o.x, y + o.y}; }
    PT operator-(PT o) const { return {x - o.x, y - o.y}; }
    PT operator*(ld k) const { return {x * k, y * k}; }
    PT operator/(ld k) const { return {x / k, y / k}; } };
ld dot(PT a, PT b){ return a.x * b.x + a.y * b.y; }
ld cross(PT a, PT b){ return a.x * b.y - a.y * b.x; }
ld norm(PT a){ return hypotl(a.x, a.y); }
ld dist(PT a, PT b){ return norm(a - b); }
PT perp(PT a){ return {-a.y, a.x}; }                                  // rota 90 antihorario
PT rotate(PT a, ld t){ return {a.x * cosl(t) - a.y * sinl(t), a.x * sinl(t) + a.y * cosl(t)};
}
ld angleBetween(PT a, PT b){ return fabsl(atan2l(cross(a, b), dot(a, b))); }  // [0, pi]
PT polar(ld r, ld t){ return {r * cosl(t), r * sinl(t)}; }
```

[GF2] Rectas y segmentos — Una recta se da con dos puntos a, b.

```cpp
// Rectas y segmentos (recta = dos puntos a, b)
// Requiere: flotantes/punto.cpp
PT project(PT p, PT a, PT b){ PT d = b - a; return a + d * (dot(p - a, d) / dot(d, d)); }
PT reflect(PT p, PT a, PT b){ return project(p, a, b) * 2 - p; }
ld distLine(PT p, PT a, PT b){ return fabsl(cross(b - a, p - a)) / norm(b - a); }
ld distSeg(PT p, PT a, PT b){ PT d = b - a; ld t = dot(p - a, d) / dot(d, d);
    t = max((ld)0, min((ld)1, t)); return dist(p, a + d * t); }
bool lineInter(PT a, PT b, PT c, PT d, PT &out){ // rectas ab y cd; false si paralelas
    ld den = cross(b - a, d - c); if (sgn(den) == 0) return false;
    out = a + (b - a) * (cross(c - a, d - c) / den); return true; }
```

[GF3] Círculos — Devuelven 0, 1 o 2 puntos. Trampa: `circleCircle` con dos círculos idénticos devuelve vacío (infinitos puntos): revísalo aparte.

```cpp
// Circulos
// Requiere: flotantes/rectas.cpp, reales/funciones_seguras.cpp
vector<PT> circleLine(PT c, ld r, PT a, PT b){ // puntos de corte circulo-recta
    PT p = project(c, a, b); ld h2 = r*r - dot(p - c, p - c);
    if (h2 < -EPS) return {}; PT d = (b - a) / norm(b - a); ld h = safeSqrt(h2);
    if (h < EPS) return {p}; return {p - d * h, p + d * h}; }
vector<PT> circleCircle(PT c1, ld r1, PT c2, ld r2){ // circulos distintos
    ld d = dist(c1, c2); if (d > r1 + r2 + EPS || d < fabsl(r1 - r2) - EPS || d < EPS) return
{};
    ld a = (r1*r1 - r2*r2 + d*d) / (2*d), h = safeSqrt(r1*r1 - a*a);
    PT m = c1 + (c2 - c1) * (a / d), off = perp(c2 - c1) * (h / d);
    if (h < EPS) return {m}; return {m - off, m + off}; }
PT circumcenter(PT a, PT b, PT c){ // centro del circulo por 3 puntos (no colineales)
    PT B = b - a, C = c - a; ld D = 2 * cross(B, C);
    return a + PT{C.y * dot(B, B) - B.y * dot(C, C), B.x * dot(C, C) - C.x * dot(B, B)} / D;
}
ld circleInterArea(ld r1, ld r2, ld d){ // area de la interseccion de dos circulos
    if (d >= r1 + r2) return 0; if (d <= fabsl(r1 - r2)){ ld r = min(r1, r2); return PI * r *
r; }
    ld a1 = safeAcos((d*d + r1*r1 - r2*r2) / (2*d*r1)), a2 = safeAcos((d*d + r2*r2 - r1*r1) /
(2*d*r2));
    return r1*r1*(a1 - sinl(2*a1)/2) + r2*r2*(a2 - sinl(2*a2)/2); }
```

Relación de dos círculos con distancia d entre centros: separados si d > r1+r2; tangentes exteriores si d = r1+r2; se cortan si |r1−r2| < d < r1+r2; uno dentro del otro si d < |r1−r2|.

[GF4] Polígonos — Vértices en orden (horario o antihorario).

```cpp
// Poligonos
// Requiere: flotantes/punto.cpp
ld polyArea(vector<PT>& p){ ld s = 0; int n = p.size(); for (int i = 0; i < n; i++) s +=
cross(p[i], p[(i+1)%n]); return fabsl(s) / 2; }
PT centroid(vector<PT>& p){ ld A = 0; PT c = {0, 0}; int n = p.size();
    for (int i = 0; i < n; i++){ ld w = cross(p[i], p[(i+1)%n]); A += w; c = c + (p[i] +
p[(i+1)%n]) * w; }
    return c / (3 * A); }
int inPolygon(vector<PT>& p, PT q){ // 1 dentro, 0 en el borde, -1 fuera
    int n = p.size(); bool in = false;
    for (int i = 0; i < n; i++){ PT a = p[i], b = p[(i+1)%n];
        if (sgn(cross(b - a, q - a)) == 0 && sgn(dot(a - q, b - q)) <= 0) return 0;
        if ((a.y > q.y) != (b.y > q.y) && q.x < a.x + (b.x - a.x) * (q.y - a.y) / (b.y -
a.y)) in = !in; }
    return in ? 1 : -1; }
// poligono regular de n lados y lado s: area = n*s*s / (4*tan(PI/n)); circunradio R = s /
(2*sin(PI/n)); apotema = s / (2*tan(PI/n))
ld regularArea(int n, ld s){ return n * s * s / (4 * tanl(PI / n)); }
```

Polígono regular de n lados	Fórmula
Ángulo interior	(n − 2)·180° / n
Circunradio (centro a vértice)	R = s / (2·sin(π/n))
Apotema (centro a lado)	s / (2·tan(π/n))
Área con circunradio R	n·R²·sin(2π/n) / 2
Diagonales	n·(n − 3) / 2
Triángulos con vértices del polígono	C(n, 3) → [CB2] si n es enorme

Áreas y volúmenes

[VO1] Volúmenes — Requiere [F1].

```cpp
// Volumenes y areas
// Requiere: reales/base_flotante.cpp
ld cone(ld r, ld h){ return PI * r * r * h / 3; }
ld frustum(ld r, ld R, ld h){ return PI * h * (R*R + r*r + R*r) / 3; }  // cono truncado
ld cylinder(ld r, ld h){ return PI * r * r * h; }
ld sphere(ld r){ return 4 * PI * r * r * r / 3; }
ld sphereArea(ld r){ return 4 * PI * r * r; }
ld cap(ld r, ld h){ return PI * h * h * (3 * r - h) / 3; }              // casquete de
altura h
ld pyramid(ld baseArea, ld h){ return baseArea * h / 3; }
```

Figura	Área / volumen
Círculo	área πr², perímetro 2πr
Sector de ángulo t (rad)	área r²·t/2, arco r·t
Segmento circular de ángulo t	r²·(t − sin t)/2
Elipse de semiejes a, b	π·a·b
Trapecio	(b1 + b2)·h / 2
Prisma / cilindro	área de la base · altura
Cono / pirámide	área de la base · altura / 3
Superficie lateral del cono	π·r·g, g = √(r² + h²)
Caja con huecos verticales (cilindros)	(N·M − Σ πrᵢ²) · altura

[VO2] Altura de llenado — Cuándo: "¿hasta qué altura llenar para tener la fracción f del volumen?". Patrón general: escribe vol(y) = volumen hasta la altura y (creciente) y aplícale [F3]. Probado con los 4 ejemplos de UCAB 2025 C (Café).

```
// Altura de llenado de una taza (cono truncado, r abajo, R arriba) hasta fraccion f
// Requiere: volumenes/volumenes.cpp, reales/busqueda_binaria_real.cpp
ld fillHeight(ld r, ld R, ld h, ld f){
    auto vol = [&](ld y){ ld ry = r + (R - r) * y / h; return frustum(r, ry, y); };
    return bsReal(0, h, f * vol(h), vol); }
```

Trampa: el "nivel de agua al meter objetos" es volumen / área libre de la base; compara `V >= A * areaLibre` en `long double` en vez de dividir.

Tests: `tests/test_enteros.cpp`, `tests/test_flotantes.cpp` (incluye los ejemplos de UCAB 2025 C y LAR 2024 J).

Esqueletos de problema completos

Copia el que se parezca a la entrada y rellena el centro con la plantilla que toque.

Archivo	Cuándo
`t1_n_y_lista.py`	"N y una lista", con o sin varios casos
`t2_leer_todo.py`	Entrada grande o con formato raro: leer todo de golpe
`t3_grafo_bfs.cpp`	"Grafo con N nodos y M aristas"
`t4_varios_casos.cpp`	Varios casos de prueba en C++
`t5_interactivo.cpp`	Problemas interactivos (preguntar al juez)
`t6_stress.sh` + `gen.py` + `brute.py`	Encontrar el caso que rompe tu solución

[T1] Python — "N y una lista" (con o sin varios casos)

```
import sys
input = sys.stdin.readline

def main():
    t = int(input())                # quita estas 2 lineas si hay un solo caso
    for _ in range(t):
        n = int(input())
        a = list(map(int, input().split()))
        print(sum(a))

main()
```

[T2] Python — leer todo de golpe (entrada grande o formato raro)

```
import sys
data = sys.stdin.buffer.read().split()   # todo el input de golpe: lo mas rapido
pos = 0
def nxt():
    global pos
    pos += 1
    return data[pos - 1]
n, m = int(nxt()), int(nxt())
g = [[] for _ in range(n)]
for _ in range(m):
    u, v = int(nxt()) - 1, int(nxt()) - 1
    g[u].append(v); g[v].append(u)
```

Trampa: `data` contiene bytes; para strings usa `nxt().decode()`.

[T3] C++ — "grafo con N nodos y M aristas" (BFS desde el nodo 1)

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
int main(){
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m; cin >> n >> m;
    vector<vector<int>> adj(n);
    for (int i = 0; i < m; i++){ int u, v; cin >> u >> v; u--; v--; adj[u].push_back(v);
adj[v].push_back(u); }
    vector<int> d(n, -1); queue<int> q; d[0] = 0; q.push(0);
    while (!q.empty()){ int u = q.front(); q.pop();
        for (int v : adj[u]) if (d[v] == -1){ d[v] = d[u] + 1; q.push(v); } }
    for (int i = 0; i < n; i++) cout << d[i] << " \n"[i == n - 1];
}
```

[T4] C++ — varios casos de prueba — Trampa: reinicia TODAS las estructuras globales dentro del bucle.

```
int main(){
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int t; cin >> t;
    while (t--){
        int n; cin >> n;
        vector<ll> a(n); for (auto &x : a) cin >> x;
        cout << accumulate(a.begin(), a.end(), 0LL) << "\n";
    }
}
```

[T5] Interactivo (búsqueda binaria contra el juez) — Trampa: vacía el búfer tras CADA mensaje (`endl` o `cout.flush()` en C++, `print(..., flush=True)` en Python); no uses `sync_with_stdio(false)` sin hacer flush.

```
int main(){
    int lo = 1, hi = 1000000;
    while (lo < hi){
        int mid = lo + (hi - lo) / 2;
        cout << "? " << mid << endl;        // endl SI aqui: hace flush
        string r; cin >> r;                 // formato de respuesta segun el enunciado
        if (r == "<") hi = mid; else lo = mid + 1;
    }
    cout << "! " << lo << endl;
}
```

[T6] Probar contra fuerza bruta (cuando da WA y no ves por qué) — Escribe `brute.py` lento pero obvio, un generador de casos pequeños, y compara.

```
for i in $(seq 1 500); do
  python3 gen.py $i > in.txt
  ./A < in.txt > o1.txt; python3 brute.py < in.txt > o2.txt
  if ! cmp -s o1.txt o2.txt; then echo "FALLA"; cat in.txt; break; fi
done
```

Cómo usar el stress test

1. Escribe tu solución en `A.cpp` y compílala como `./A`.
2. Escribe en `brute.py` una solución lenta pero obviamente correcta (fuerza bruta).
3. Ajusta `gen.py` para que genere entradas **pequeñas** con el formato del problema.
4. Corre `./t6_stress.sh`: si encuentra una diferencia, imprime el caso que falla.

Estrategia, checklist del juez y mapa de problemas

Cómo reconocer qué técnica pide un problema, qué revisar antes de enviar y cómo repartir las 5 horas.

[J1] Antes de CADA envío (cada rechazo cuesta 20 minutos de penalización)

1. El archivo se llama como la letra del problema: `A.cpp`, `A.java`, `A.py3`. En Java/Kotlin: sin `package` y la clase pública se llama `A`.

2. Sin `cerr`/prints de depuración: stderr cuenta en el límite de salida (~1 MB) y puede dar Runtime Error.
3. Formato exacto: un espacio entre valores, `\n` al final de cada línea, sin líneas vacías extra (WA vs Presentation Error).
4. `ll` donde haya sumas, productos o distancias. ¿El peor caso desborda `int`?
5. Probaste: ejemplos del enunciado + N mínimo (1) + N máximo + todos iguales + negativos/ceros si se permiten.
6. Complejidad vs límites de la tabla del índice. Si es Python con N ≥ 10^5: pásalo a C++.
7. Decimales con la cantidad de cifras pedida (`setprecision`, `:.Nf`).
8. Interactivo: flush después de cada línea.

[J2] Veredicto → qué revisar

Veredicto	Revisa primero
Wrong Answer	Desborde, caso N = 1, empates, índices 0/1, inicialización entre casos
Time Limit	Complejidad, Python → C++, `endl`, I/O rápido, bucle infinito
Runtime Error	Índice fuera de rango, división por 0, recursión profunda, arreglo enorme local, salida en stderr
Presentation Error	Espacios al final, líneas vacías, formato de decimales
Name/Problem mismatch	Nombre del archivo o de la clase

[J3] Estrategia de 5 horas (meta Top 10 SA-Norte = 4–5 problemas)

Tiempo	Qué hacer
0:00–0:20	Repartir los 12–13 enunciados entre los 3; cada uno marca fácil / medio / difícil
Todo el concurso	Mira el scoreboard: los problemas con más YES en el continente son los fáciles
0:20–1:30	Asegurar 2–3 fáciles (ad-hoc, simulación, strings, greedy). Python si cabe
1:30–4:00	Bloque medio: grafos [G], constructivos [M6], DP, estructuras [D]
4:00–5:00	Si el marcador se congela, sigue con el problema más avanzado; no abras uno nuevo en los últimos 30 min sin idea clara

Regla de equipo: una persona en el teclado, las otras dos piensan en papel; si llevas 30 min atascado en un problema, cambia de problema o de persona.

Señales en el enunciado → plantilla

Si el enunciado dice…	Piensa en…	Ir a
"mínimo/máximo valor tal que se pueda"	Búsqueda binaria sobre la respuesta	[A3]
"subarreglo/subcadena más larga que cumple"	Dos punteros o prefijos + map	[A4] [A1]
"camino más corto", pesos 1	BFS	[G2] [G3]
"camino más corto", pesos variables ≥ 0	Dijkstra	[G5]
"todos los caminos óptimos pasan por X"	Dijkstra con y sin X	[G5]
"qué nodos pueden llegar a X"	Invertir aristas + BFS desde X	[G6]
"tareas con dependencias", sin ciclos	Topo + DP en DAG	[G7]
"unir grupos", "¿están conectados?"	DSU	[G8]
"quitar aristas sin desconectar", "ciclo"	Árbol DFS, back-edges, puentes	[G9]
Árbol + "distancia entre u y v" / "ancestro"	LCA	[G10]
Árbol + "para cada nodo como raíz"	Rerooting	[G11]
"emparejar", "asignar sin repetir"	Matching bipartito	[G12]
"capacidad", "cortar mínimo"	Flujo / corte mínimo	[G13]
Muchas consultas de suma/min en rango	Prefijos, Fenwick, segment tree, sparse	[A1] [D1] [D3] [D4]
"k-ésimo disponible/menor"	Fenwick `kth` u ordered set	[D1] [D2]
max de $A_i \cdot X + B_i$	Li-Chao / CHT	[D5]
Actualizaciones que fuerzan recalcular todo	Sqrt batching	[D6]
"igual frecuencia", comparar subcadenas	Rolling hash	[S1]
"expresión válida", paréntesis	Reducción canónica + balance	[S4] [S5]
"módulo 10^9+7"	Aritmética modular	[M1]
N hasta 10^12 y divisores/bases	O(√N)	[M2] [M3]
"k-ésima permutación con…"	DP de conteo + reconstrucción	[M4]
"dos jugadores, juego óptimo"	Invariante o Grundy	[M5]
"construye", "rellena los ceros"	Pensar hacia atrás, extremos, bloques	[M6]
Puntos y polígonos	Cruz, hull, ángulos	[GE]

Señales de matemática y geometría

Si el enunciado dice…	Ir a
"error absoluto o relativo", salida con decimales	[F1] [F2]
"hasta qué altura / radio / tiempo para que…"	[F3] (+ [VO2])
"máximo/mínimo de una función continua" que sube y baja	[F4]
Ángulo en grados, luz, sombra, pendiente	[TR1] [TR2]
Triángulo dado por sus lados	[TR2]
Girar puntos	[TR3]
Distancia de un punto a una recta / segmento	[GF2]
Círculos que se cortan, punto dentro de círculo	[GF3]
Punto dentro de un polígono, área, centro de masa	[GF4]
Cono, esfera, cilindro, taza, tanque	[VO1] [VO2]
"longitud total cubierta"	[IV1]
n hasta 10^18 y "módulo"	[CB1] [CB2]
Módulo como 10^18+3 o producto de dos ≤ 10^18	[CB1]
"al menos uno de…", "ninguno de…"	[CB5]
"valor esperado", dados, monedas, "repetir hasta"	[PR1] [PR3]
"probabilidad módulo p"	[PR2]
a·x + b·y = c, "combinar paquetes de a y b"	[NT2]
"resto r1 al dividir entre m1 y r2 entre m2"	[NT4]
Divisores de n hasta 10^12, perfecto/abundante	[NT6]
"¿es primo?" con n hasta 10^18	[NT7]
Recurrencia con n hasta 10^18 (Fibonacci y similares)	[NT8]
Σ de n / i	[NT10]
Intercambios, mezclas repetidas, "volver al orden inicial"	[PE1]

Problemas del Regional Latinoamericano por técnica

[R2] Problemas del Regional Latinoamericano por tema

Año	Problema	Técnica	Complejidad	Plantillas
2022-23	A. Asking for Money	Inversión de aristas + 3 BFS/DFS	$O(N^2)$	[G6]
2023	E. Elevated Profits	Eliminación inversa de hojas + buckets por promedio	O(N log N)	[M6] [C4]
2023	M. Meeting Point	Dijkstra dos veces, omitiendo un nodo	O(M log N)	[G5]
2024	B. Biketopia's Cyclic Track	Árbol DFS + back-edge de costo máximo	O(N + M)	[G9]
2024	D. Diverse T-Shirts	Cliques maximales en grafo línea + matching bipartito	O(V·E)	[G12]
2024	G. Grand Glory Race	Rerooting (mejor y segundo mejor) + LCA y saltos	$O(N \log N + Q \log^2 N)$	[G11] [G10]
2024	H. Heraclosures	DP en DAG por orden topológico + sqrt batching	O(NM + E√M)	[G7] [D6]
2022-23	C. City Folding	Simulación inversa de pliegues	O(N)	[M6]
2023	B. Blackboard Game	Invariante: frecuencias módulo 3	O(N)	[M5]
2023	F. Forward and Backward	Divisores y bases palindrómicas	O(√N)	[M2] [M3]
2024	E. Evereth Expedition	Dos punteros en permutación bitónica	O(N)	[M6]

Año	Problema	Técnica	Complejidad	Plantillas
2024	F. Finding Privacy	Sistema diofántico lineal + bloques greedy	O(N)	[M6]
2024	I. Inversion Insight	DP con sumas prefijas + order statistics	O(N·X)	[M4] [D1] [D2]
2023	C. Candy Rush	Rolling hash dinámico de frecuencias	O(N log N)	[S1]
2023	D. Deciphering WordWhiz	Simulación directa sobre diccionario	O(N·Q)	[P3]
2023	L. Latam++	Reducción canónica + paréntesis	O(N)	[S5] [S4]
2024	A. Append and Panic!	Conteo con set / barrido inverso	O(|S|)	[P5]

Año	Problema	Técnica	Complejidad	Plantillas
2022-23	B. Board Game	Segment tree + CHT / Li-Chao	O(N $\log^2$ N)	[D3] [D5]
2023	A. Analyzing Contracts	Li-Chao (cruce único) o sqrt + D&C	O(N + Q log N)	[D5] [D6]
2024	C. Cindy's Christmas Challenge	Búsqueda binaria + 16 RMQ	O(W log |S|)	[A3] [D4]

Practica estos en el gimnasio de Codeforces (Latin America Regional 2020–2025) en simulacros de 5 horas.

Los códigos entre corchetes ([G5], [D1], [CB2]...) son los mismos de los README de cada carpeta y de los PDF de `notebook/`.